THE ZERO TO HERO COLLECTION · PREMIUM SERIES

Spring Boot with Kafka Interview Notes

The clean handwritten-style edition

From your first Kafka listener to production-ready reasoning about offsets, transactions, retries and failure recovery.

100 unchanged interview questions

100 solved follow-ups · 20 topic chapters

10 practical exercises · 30-day study plan



Java 17+ · Spring Boot 3.5 · Spring Kafka 3.3

Designed for freshers, Java developers, team leads and technical architects.

PUBLISH · PROCESS · RECOVER

How to use these notes

Read the question. Say the short answer aloud. Trace the example and the visual memory map. Then explain the failure window before checking the follow-up answer.

Answer in interviews: Business rule → owner → mechanism → possible crash → recovery → verification.

FRESHER

Know the record, topic, partition, key, and group.

DEVELOPER

Implement listeners, retries, offsets, validation and tests.

SENIOR

Protect durable effects, schema evolution, operation IDs, and capacity.

ARCHITECT

Prove invariants during crash, replay, partition and rollback.

Read the guarantees precisely

Producer acknowledgment does not prove that billing completed. A committed offset does not prove the database is correct. Kafka transactions do not atomically cover every HTTP or database effect. Every chapter names the boundary it can and cannot protect.

Source fidelity

All 100 question titles, original explanations, worked examples, pitfalls, and follow-up questions come from the attached source. Follow-up answers and visual memory maps have been added for this study edition.

Version baseline: Java 17+, Spring Boot 3.5, Spring Kafka 3.3-style APIs. Some examples are teaching fragments, not complete standalone applications. Check exact dependency compatibility before production use.

YOUR STUDY MAP

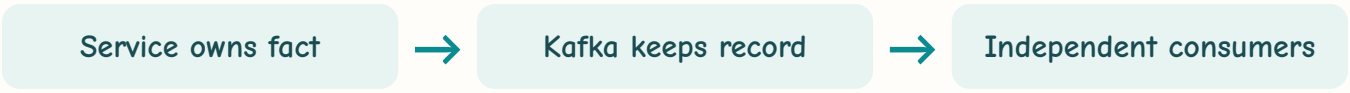
20 chapters · 100 questions

Part	Topic	Questions
01	Event-driven foundations	Q001–Q005
02	Topics, partitions and record identity	Q006–Q010
03	Replication and broker guarantees	Q011–Q015
04	Spring Boot integration	Q016–Q020
05	Producer reliability and tuning	Q021–Q025
06	Consumer groups and scaling	Q026–Q030
07	Offsets and acknowledgment	Q031–Q035
08	Poll loop and processing pressure	Q036–Q040
09	Failures, retries and dead-letter topics	Q041–Q045
10	Kafka transactions and exactly-once boundaries	Q046–Q050
11	Idempotency, outbox and business correctness	Q051–Q055
12	Serialization and schema evolution	Q056–Q060
13	Retention, compaction and replay	Q061–Q065
14	Testing Spring Boot Kafka applications	Q066–Q070
15	Observability and performance	Q071–Q075
16	Security and deployment	Q076–Q080
17	Streams and ecosystem choices	Q081–Q085
18	Architecture and sizing	Q086–Q090
19	Production troubleshooting scenarios	Q091–Q095
20	Senior design and interview challenges	Q096–Q100

The separate lab section has C01–C10. Use the 30-day roadmap at the end for revision and mock interviews.

01. Event-driven foundations

An event is a committed fact; a command requests work. Separate acceptance from completion.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

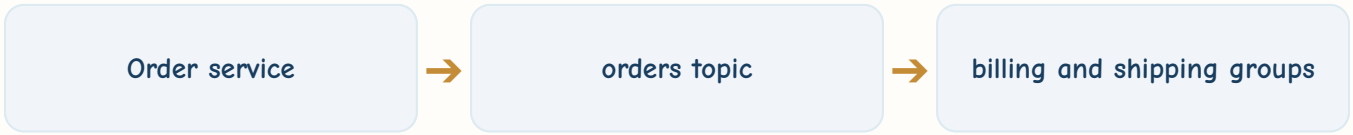
What is Kafka and why use it with Spring Boot?

THE SIMPLE ANSWER

Kafka is a distributed event log; Spring Boot supplies the application wiring for publishing and consuming records.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Order service -> orders topic -> billing and shipping groups

WHY IT WORKS

One published order event can feed independent services. Consumers keep their own progress and can replay retained records. Kafka adds asynchronous delivery, but the application still owns business correctness.

COMMON INTERVIEW TRAP

Kafka is not a substitute for a database transaction.

FOLLOW-UP QUESTION

When would a direct REST call be simpler?

Choose REST when the caller must receive a small, immediate answer, and the target service is available within the request deadline. Example: checking an address before submitting a form.

How does an event differ from a command?

THE SIMPLE ANSWER

An event states something that happened; a command asks a specific capability to do something.

VISUAL MEMORY MAP

Read left to right

Service owns fact → Kafka keeps record → Independent consumers

CODE / WORKED EXAMPLE

```
Event: OrderPlaced(orderId=42)
   Command: ReserveInventory(orderId=42)
```

WHY IT WORKS

An order event should describe a committed fact. A reservation command can be rejected and needs an outcome. Naming and ownership prevent several consumers from accidentally treating the same request as their responsibility.

COMMON INTERVIEW TRAP

Publishing an event before the underlying fact commits can mislead consumers.

FOLLOW-UP QUESTION

Who owns rejection of a command?

The service that owns the requested capability decides whether to accept or reject it. Inventory rejects ReserveInventory when stock is insufficient and publishes or returns a documented result.

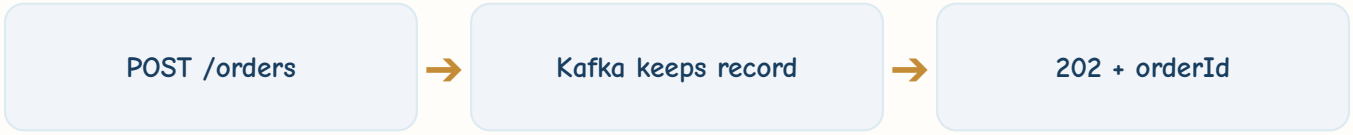
When should Kafka replace synchronous REST?

THE SIMPLE ANSWER

Use Kafka when independent processing, buffering or replay matters more than an immediate response from the downstream service.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
POST /orders -> 202 + orderId
  GET /orders/42 -> PENDING, CONFIRMED or REJECTED
```

WHY IT WORKS

The HTTP response confirms the documented acceptance boundary, not completion of every consumer. Expose an explicit status model and failure path. Use synchronous calls where the caller genuinely needs the downstream result now.

COMMON INTERVIEW TRAP

A fast HTTP response does not prove the event was durably accepted.

FOLLOW-UP QUESTION

What does 202 mean in your API?

HTTP 202 means the request was accepted for later handling, not that all business steps finished. Return an operation ID and offer a status endpoint showing PENDING, COMPLETED or FAILED.

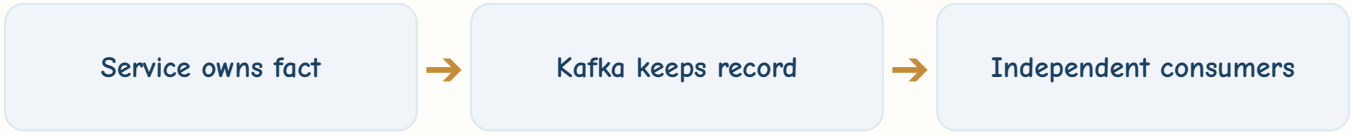
How is Kafka different from a traditional work queue?

THE SIMPLE ANSWER

Kafka retains a partitioned log and tracks progress per consumer group instead of deleting each record after one consumer receives it.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
orders: offsets 0,1,2,3 remain within retention
   billing group next=4; analytics group next=2
```

WHY IT WORKS

Within a group, partition ownership distributes work. Across groups, each application can read the same records independently. Retention is a storage policy, so a slow consumer can fall behind the available history.

COMMON INTERVIEW TRAP

Consumption does not reset the retention clock.

FOLLOW-UP QUESTION

How would you detect data lost to retention?

Track each group's oldest available offset against its next committed offset. Alert when retention advances past unprocessed data; compare processing history and recover missing data from a trusted source.

What is eventual consistency in an event-driven system?

THE SIMPLE ANSWER

Related services can temporarily show different states while events travel and are processed.

VISUAL MEMORY MAP

Read left to right

Service owns fact → Kafka keeps record → Independent consumers

WORKED EXAMPLE

Order: CONFIRMED at 10:00:00
Search projection: PENDING until 10:00:02

WHY IT WORKS

Define an acceptable lag, user-visible pending state and reconciliation process. A broken consumer is not automatically eventual consistency: without recovery the projection may never converge.

COMMON INTERVIEW TRAP

Do not promise immediate reads from an asynchronous projection.

FOLLOW-UP QUESTION

What convergence metric would you monitor?

Measure time from authoritative event commit to successful projection update, especially the oldest pending event age. Alert on exceeded freshness SLOs and on projections that stop advancing.

02. Topics, partitions and record identity

Identity uses topic + partition + offset. Ordering is per partition, not global.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

What are a topic, partition and offset?

THE SIMPLE ANSWER

A topic groups records; partitions are ordered logs; offsets identify positions within one partition.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
orders-0 offset 19 != orders-1 offset 19
  Record identity: topic + partition + offset
```

WHY IT WORKS

Offsets are assigned by the broker and are not global sequence numbers. Compaction and transactions can leave gaps in visible records. Use a separate eventId for business deduplication across republishing.

COMMON INTERVIEW TRAP

An offset alone is not globally unique.

FOLLOW-UP QUESTION

Why can a consumer see offset gaps?

Offsets can be absent from what a consumer sees because of compaction, aborted transactions under read_committed, retention, or filtering. Do not assume every numeric offset has a visible record.

Where does Kafka guarantee ordering?

THE SIMPLE ANSWER

Kafka preserves log order within a partition, not a global order across partitions.

VISUAL MEMORY MAP

Read left to right

customer A → Partition log → partition 0: Created, Updated

WORKED EXAMPLE

```
customer A -> partition 0: Created, Updated
  customer B -> partition 1: Created, Updated
```

WHY IT WORKS

Choose a stable aggregate key when updates for that aggregate must share a partition. Consumer worker pools, retry topics and application logic can still reorder side effects after records are read.

COMMON INTERVIEW TRAP

Partition order does not guarantee ordered asynchronous completion.

FOLLOW-UP QUESTION

How would you preserve account-level processing order?

Send all events for an account with the same stable account ID as key, process each partition in sequence, and avoid asynchronous handlers that finish out of order. Repartitioning requires a migration plan.

How does a record key affect partition selection?

THE SIMPLE ANSWER

The producer partitioner uses the key and partition metadata unless an explicit partition or custom partitioner changes that choice.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
template.send("orders", order.customerId(), json);
```

WHY IT WORKS

A stable key normally routes related events to the same partition for a fixed topic layout. Null keys follow the configured partitioner behavior. Do not rely on an undocumented hand-written modulo formula to match every client version.

COMMON INTERVIEW TRAP

A business key can create a hot partition.

FOLLOW-UP QUESTION

Which key would you choose for a payment ledger?

Use the payment or account identity whose operations must stay ordered (for example accountId), not a random eventId. Consider hot accounts and make the partitioning policy explicit.

What happens when a topic gains partitions?

THE SIMPLE ANSWER

It increases possible parallelism, but key-to-partition mapping can change for future records.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Before: account-7 -> partition 1
After expansion: account-7 may -> partition 4

WHY IT WORKS

Older records remain in their original partitions. If a key moves, strict ordering across old and new history needs an explicit migration strategy, drain or versioned topic. Partition reduction is not a normal in-place operation.

COMMON INTERVIEW TRAP

Adding partitions is not a harmless ordering change.

FOLLOW-UP QUESTION

How would you migrate an ordered topic?

Create a versioned topic with the desired partition count, establish a controlled cutover point, drain old work, and route new writes to the new topic. Reconcile versions and consumer progress before retirement.

Are timestamp and offset the same?

THE SIMPLE ANSWER

A timestamp describes record time semantics; an offset describes position in a partition log.

VISUAL MEMORY MAP

Read left to right

Business key → Partition log → Offset and consumer

WORKED EXAMPLE

partition 0: offset 20 timestamp 10:05
partition 0: offset 21 timestamp 10:03

WHY IT WORKS

Producer create times can be out of order because clocks and sources differ. Broker append-time policy is a different choice. Event time used in business processing should be explicit in the event contract.

COMMON INTERVIEW TRAP

Sorting across topics by timestamp does not establish causality.

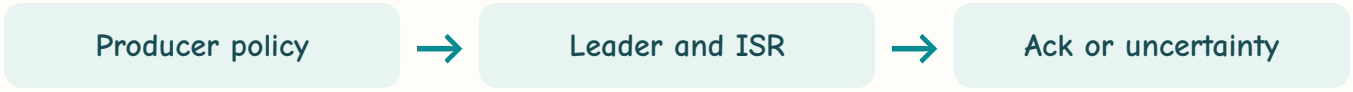
FOLLOW-UP QUESTION

How would you handle late events?

Define event-time semantics and acceptable lateness. Buffer or update time-window results until a grace limit; after that use a correction event, reconciliation or a documented late-data policy.

03. Replication and broker guarantees

Replication changes which broker failures you survive. Acks alone do not prove business completion.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

What are leader, follower and ISR?

THE SIMPLE ANSWER

The leader handles partition writes; followers replicate; ISR is the set of replicas currently sufficiently caught up.

VISUAL MEMORY MAP

Read left to right

Producer policy → Leader and ISR → Ack or uncertainty

CODE / WORKED EXAMPLE

```
Replication factor=3; ISR={broker1, broker2}
   Leader=broker1; broker3 is lagging
```

WHY IT WORKS

ISR membership changes as replicas fail or catch up. Replication factor is the desired replica count, while ISR describes current replication health. Observe under-replicated partitions and shrinking ISR during incidents.

COMMON INTERVIEW TRAP

Replication factor three does not mean three healthy copies right now.

FOLLOW-UP QUESTION

What metric would trigger your durability alert?

Alert on under-replicated partitions, offline replicas, ISR shrinkage and partitions below min.insync.replicas. Include broker and availability-zone health so an alert signals actual durability risk.

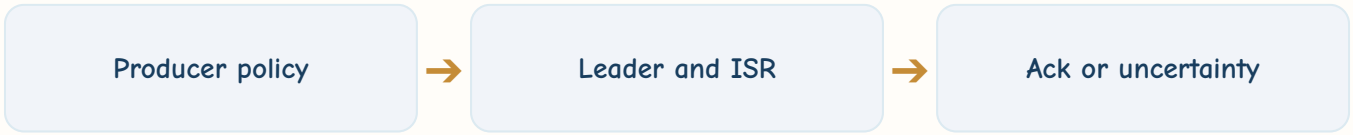
What do acks=0, acks=1 and acks=all mean?

THE SIMPLE ANSWER

They choose no broker acknowledgment, leader acknowledgment, or acknowledgment from the current in-sync replica set.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
acks=0: no response confirmation
   acks=1: leader confirms
   acks=all: ISR acknowledgment requirement
```

WHY IT WORKS

acks=all is not acknowledgment from every configured replica, including lagging ones. It works with min.insync.replicas to enforce a minimum healthy replica count for accepted writes.

COMMON INTERVIEW TRAP

Acknowledgment does not mean a consumer processed the record.

FOLLOW-UP QUESTION

What can still fail after acks=1 succeeds?

The leader can acknowledge before followers copy the record. If the leader then fails and a different replica wins, the acknowledged record can be lost under relevant recovery settings.

How do min.insync.replicas and acks=all work together?

THE SIMPLE ANSWER

The broker rejects all-ack writes when the ISR falls below the configured minimum.

VISUAL MEMORY MAP

Read left to right

Producer policy → Leader and ISR → Ack or uncertainty

CODE / WORKED EXAMPLE

```
replication.factor=3
   min.insync.replicas=2
   acks=all
```

WHY IT WORKS

With one in-sync replica remaining, this policy sacrifices write availability rather than accepting under-replicated writes. The exact failure can appear during or after append, so the producer must handle uncertain outcomes and retries.

COMMON INTERVIEW TRAP

min.insync.replicas alone does not enforce this producer policy for acks=1.

FOLLOW-UP QUESTION

Why might a timeout still mean the record was appended?

The broker might append the record, but its acknowledgment may be delayed or lost. Treat the response as unknown and retry using a compatible idempotent producer plus stable business identity.

Why can a successful send still have durability limits?

THE SIMPLE ANSWER

A send confirms the configured broker acknowledgment boundary, not protection against every correlated failure.

VISUAL MEMORY MAP

Read left to right

RF=3 on one physical host → Leader and ISR → one host failure loses availability

WORKED EXAMPLE

RF=3 on one physical host -> one host failure loses availability

WHY IT WORKS

Replica placement, fault domains, recovery policy and storage health matter. Kafka replication does not make application retries or external side effects automatically safe. Design for realistic rack or zone failures.

COMMON INTERVIEW TRAP

Three replicas in the same failure domain are weak protection.

FOLLOW-UP QUESTION

What failure would your deployment survive?

With three replicas across independent zones and suitable ISR/minimum settings, one broker or zone failure may be tolerated. Correlated failure, quorum loss or misplacement remain outside that guarantee.

What is KRaft and how does it relate to ZooKeeper?

THE SIMPLE ANSWER

KRaft manages Kafka cluster metadata through a controller quorum; older Kafka deployments used ZooKeeper.

VISUAL MEMORY MAP

Read left to right

Controller quorum → Leader and ISR → metadata decisions

WORKED EXAMPLE

```
Controller quorum -> metadata decisions
  Partition replicas -> record storage
```

WHY IT WORKS

Kafka 4.x is KRaft-only. Controllers and partition leaders have different responsibilities. For an interview, separate the broker/client version from the Spring Boot library version and ask which deployment generation is in scope.

COMMON INTERVIEW TRAP

Do not prescribe ZooKeeper for a new Kafka 4.x cluster.

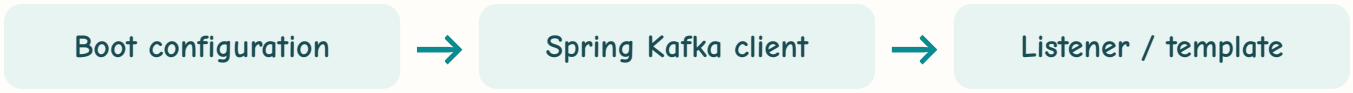
FOLLOW-UP QUESTION

How does controller loss differ from a partition leader failure?

Controller loss affects metadata leadership and cluster coordination until controller quorum recovers. Partition-leader loss affects writes/reads for that partition until an eligible replica takes leadership.

04. Spring Boot integration

Spring provides producer and listener abstractions. Your code still owns correct outcomes.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

Which dependency enables Spring Kafka integration?

THE SIMPLE ANSWER

Add spring-kafka and let the Spring Boot dependency management select compatible library versions.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
  </dependency>
```

WHY IT WORKS

The Boot parent or BOM manages versions. Broker binaries are a separate deployment concern. This guide targets Boot 3.5 and Spring Kafka 3.3 style APIs; check your resolved dependency tree before copying a version-specific configuration.

COMMON INTERVIEW TRAP

Mixing arbitrary client and framework versions can break compatibility.

FOLLOW-UP QUESTION

How do you inspect the effective dependency versions?

Run `mvn dependency:tree` (or Gradle dependencyInsight) and inspect the resolved `spring-kafka`, `kafka-clients` and Boot versions. Use the chosen Boot BOM as the compatibility baseline.

What does Boot auto-configure for Kafka?

THE SIMPLE ANSWER

It can configure producer and consumer factories, KafkaTemplate, administration and a default listener container factory from properties and available beans.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.bootstrap-servers=localhost:9092
   spring.kafka.consumer.group-id=orders-worker
```

WHY IT WORKS

Properties supply client settings; listener annotations supply application behavior. Custom beans can cause parts of auto-configuration to back off. Inspect conditions when a template or factory is unexpectedly absent.

COMMON INTERVIEW TRAP

Boot does not provision a production broker cluster.

FOLLOW-UP QUESTION

What would a custom ConsumerFactory override?

A custom ConsumerFactory replaces or customizes consumer construction, including deserializers, authentication, bootstrap addresses and group settings. Confirm the listener factory uses that bean.

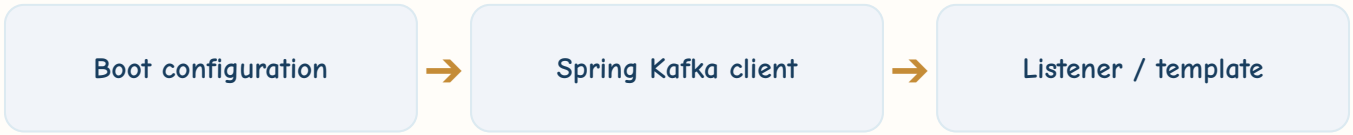
What is KafkaTemplate?

THE SIMPLE ANSWER

KafkaTemplate is Spring's publishing abstraction over the Kafka producer.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
CompletableFuture<SendResult<String,String>> future =
       template.send("orders", "42", "placed");
```

WHY IT WORKS

In modern Spring Kafka, send returns a CompletableFuture. Completion carries broker metadata or an error. A send can also fail synchronously before a future is returned, such as during serialization or unavailable metadata.

COMMON INTERVIEW TRAP

Starting a send is not the same as confirming its success.

FOLLOW-UP QUESTION

Which two failure paths must publishing code handle?

Handle a synchronous exception from `send(...)` and asynchronous exceptional completion of its `CompletableFuture`. Neither successful invocation nor a pending future is proof of broker acceptance.

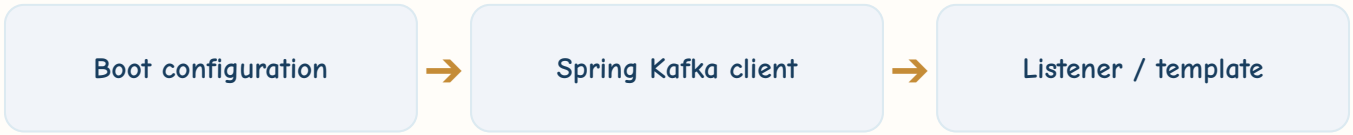
What does @KafkaListener do?

THE SIMPLE ANSWER

It declares a listener endpoint that a Spring-managed container invokes for consumed records.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
@KafkaListener(topics="orders", groupId="billing-v1")
   public void receive(String value) { service.apply(value); }
```

WHY IT WORKS

The container owns polling, dispatch and configured offset/error behavior. A bean method is needed for endpoint registration. The listener should finish the required durable work before the container treats the record as successfully handled.

COMMON INTERVIEW TRAP

A swallowed exception can make failed work appear successful.

FOLLOW-UP QUESTION

When should the listener throw?

Throw when the required durable business effect failed and the configured error handler should retry or quarantine the record. Do not swallow a database exception and permit offset progress.

How do you create a topic from an application?

THE SIMPLE ANSWER

Declare a NewTopic bean for KafkaAdmin when application-managed topic creation is appropriate.

VISUAL MEMORY MAP

Read left to right

Boot configuration → Spring Kafka client → Listener / template

CODE / WORKED EXAMPLE

```
@Bean NewTopic orders() {
     return TopicBuilder.name("orders")
         .partitions(6).replicas(3).build();
   }
```

WHY IT WORKS

The cluster must support the replica count and the principal must have administration permission. Production teams often provision topics separately through reviewed infrastructure configuration. Existing topics and changes need deliberate operational control.

COMMON INTERVIEW TRAP

A one-broker local cluster cannot satisfy replication factor three.

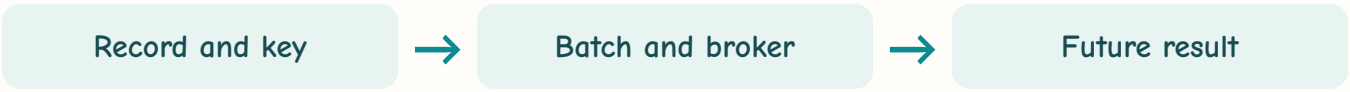
FOLLOW-UP QUESTION

Who should own topic configuration in production?

The platform or streaming team usually owns approved topic declarations, ACLs, retention and replication through infrastructure-as-code; application teams define the contract and request reviewed changes.

05. Producer reliability and tuning

Handle both synchronous send errors and asynchronous future completion.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

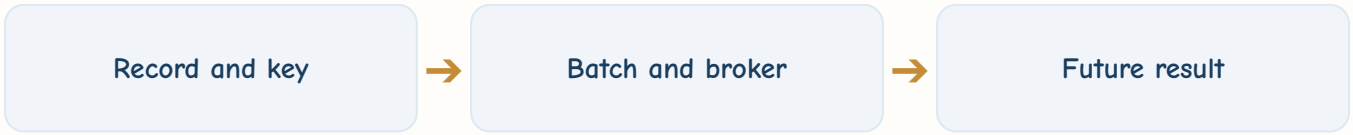
What does producer idempotence prevent?

THE SIMPLE ANSWER

It prevents duplicate log appends from eligible retries within the producer's protocol session.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.producer.acks=all
   spring.kafka.producer.properties[enable.idempotence]=true
```

WHY IT WORKS

Idempotence uses producer identity and sequence numbers. It does not deduplicate two business sends with the same eventId, or prevent a consumer from charging twice. Compatible retry and max-in-flight settings are required.

COMMON INTERVIEW TRAP

Idempotent producer does not mean idempotent business workflow.

FOLLOW-UP QUESTION

How do you deduplicate application resends?

Give the event a stable business ID, and deduplicate it at the consumer's durable write boundary (such as a unique inbox key in the same DB transaction). Producer idempotence alone cannot do this.

What is max.in.flight.requests.per.connection?

THE SIMPLE ANSWER

It limits producer requests awaiting responses on a broker connection.

VISUAL MEMORY MAP

Read left to right

Record and key → Batch and broker → Future result

WORKED EXAMPLE

Idempotent producer: supported max-in-flight <= 5

WHY IT WORKS

Without idempotence, multiple outstanding requests and retries can reorder batches. With compatible idempotence settings, ordering is preserved for supported values. Do not tune this value in isolation from latency, throughput and retry behavior.

COMMON INTERVIEW TRAP

Setting it arbitrarily high conflicts with idempotence requirements.

FOLLOW-UP QUESTION

Why might setting it to one reduce throughput?

Only one batch can be in flight, which prevents pipeline overlap. On high-latency broker links it can substantially reduce throughput; benchmark compatible idempotent settings instead.

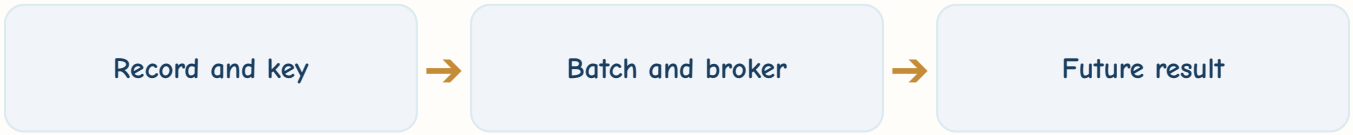
How do linger.ms and batch.size affect publishing?

THE SIMPLE ANSWER

They influence waiting and batching of records, trading latency for throughput and compression efficiency.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
linger.ms=10
   batch.size=32768
```

WHY IT WORKS

The producer batches per partition. A full batch can send before the linger interval expires. Defaults vary by Kafka client version, so measure the resolved client's behavior rather than memorizing one default forever.

COMMON INTERVIEW TRAP

batch.size is not the maximum event size.

FOLLOW-UP QUESTION

How would you benchmark latency at low traffic?

Publish small messages at controlled low rates, measure p50/p95/p99 end-to-end latency with linger set to multiple values, and record producer batch size, network use and CPU.

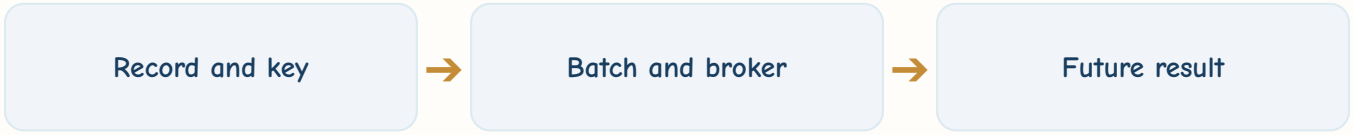
How do delivery.timeout.ms and request.timeout.ms differ?

THE SIMPLE ANSWER

Request timeout bounds an individual response wait; delivery timeout bounds the overall delivery lifecycle including retry-related time.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
delivery.timeout.ms=120000
   request.timeout.ms=30000
```

WHY IT WORKS

The delivery budget includes batching delay and retries under the producer's rules. max.block.ms separately bounds selected waits during send, such as metadata or buffer availability. Use budgets aligned with API expectations.

COMMON INTERVIEW TRAP

An HTTP timeout does not cancel a Kafka record already sent.

FOLLOW-UP QUESTION

What response should an API return for an unknown send outcome?

Return an explicit PENDING/UNKNOWN state with a stable operation ID if the business contract allows it. Reconcile the send result from durable intent; do not claim failure or success from timeout alone.

Why use compression and bounded producer buffers?

THE SIMPLE ANSWER

Compression can reduce network/storage work, while bounded buffers expose overload instead of hiding it indefinitely.

VISUAL MEMORY MAP

Read left to right

Record and key → Batch and broker → Future result

WORKED EXAMPLE

```
compression.type=zstd
  buffer.memory=33554432
```

WHY IT WORKS

Compression effectiveness depends on batches and payloads. Buffer exhaustion can block publishing up to the configured limit and then fail. Track enqueue rate, request latency and buffer wait metrics before changing memory.

COMMON INTERVIEW TRAP

Increasing buffers does not fix a permanently slow broker.

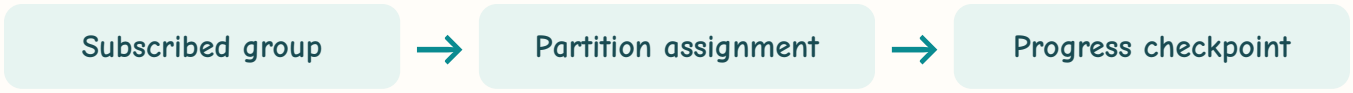
FOLLOW-UP QUESTION

How would you shed load safely?

Reject or defer excess requests with a documented 429/503 or a durable queue if the contract supports it. Bound memory and requests, protect critical workloads and expose retry-after guidance.

06. Consumer groups and scaling

Consumers in one group divide partitions; separate groups progress independently.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

What is a consumer group?

THE SIMPLE ANSWER

A group coordinates consumers that share progress and divide partition ownership for subscribed topics.

VISUAL MEMORY MAP

Read left to right

Subscribed group → Partition assignment → Progress checkpoint

WORKED EXAMPLE

orders has 6 partitions
billing group: 3 consumers -> roughly 2 partitions each
shipping group: independent progress

WHY IT WORKS

A partition is normally assigned to one consumer at a time within the group. Different groups receive independent copies of the logical stream. Assignment details depend on topics, strategy and protocol.

COMMON INTERVIEW TRAP

Consumers in different groups do not share the workload.

FOLLOW-UP QUESTION

How do you add an audit subscriber?

Create a new consumer group ID for audit, subscribe it to the same topic, grant minimum ACLs and define its offset starting policy. Billing's group progress remains independent.

How many consumers can usefully process one topic in a group?

THE SIMPLE ANSWER

Parallel partition ownership is bounded by the topic's partition count.

VISUAL MEMORY MAP

Read left to right

Subscribed group → Partition assignment → Progress checkpoint

CODE / WORKED EXAMPLE

```
6 partitions; 10 consumers in one group
   At most 6 own partitions for this topic
```

WHY IT WORKS

Extra consumers can be idle. A consumer can own several partitions, and a group can subscribe to multiple topics. Scale using measured partition load and downstream capacity, not only replica count.

COMMON INTERVIEW TRAP

More pods cannot split one hot partition between group members.

FOLLOW-UP QUESTION

How would you fix a single hot key?

First measure why the key is hot. If operations need one total order, increase per-partition processing capacity or redesign the aggregate; splitting the key sacrifices its single-partition order.

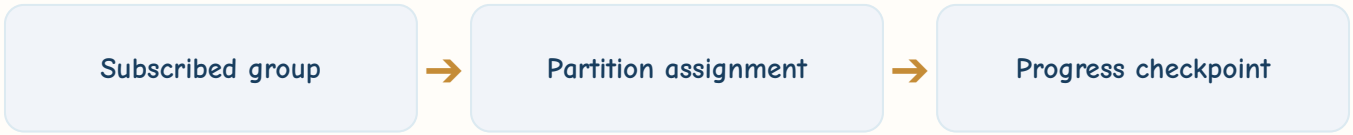
What does listener concurrency mean in Spring Kafka?

THE SIMPLE ANSWER

It configures multiple child listener containers, each with its own consumer.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
@KafkaListener(topics="orders", concurrency="3")
   public void receive(String value) { ... }
```

WHY IT WORKS

Two application instances with concurrency three create up to six consumers. Their useful assignments still depend on partitions and group strategy. Shared listener state and injected collaborators must be safe for concurrent invocation.

COMMON INTERVIEW TRAP

Concurrency is not a worker pool for one consumer instance.

FOLLOW-UP QUESTION

What state should not be stored on the listener bean?

Do not keep mutable currentOrder, currentRecord, tenantId, counters or unsynchronized maps as listener fields. Store per-record state in local variables and use safe, durable repositories.

What triggers a rebalance?

THE SIMPLE ANSWER

Membership, subscriptions or partition metadata changes can require reassignment of group partitions.

VISUAL MEMORY MAP

Read left to right

Pod joins → Partition assignment → assignment changes

WORKED EXAMPLE

Pod joins -> assignment changes
  Consumer misses progress requirements -> partition revoked

WHY IT WORKS

During ownership changes, unfinished work and commits need careful handling. Strategies and group protocols affect how much work pauses. Handle duplicates after reassignment rather than assuming an old owner finished everything.

COMMON INTERVIEW TRAP

A rebalance is not itself evidence of a broker failure.

FOLLOW-UP QUESTION

Which logs explain why membership changed?

Check consumer coordinator logs for join/leave, heartbeat expiration, max.poll.interval expiration, assignment, partitions revoked, and deployment restarts. Correlate with Kubernetes pod events.

How do cooperative assignment and static membership help?

THE SIMPLE ANSWER

They can reduce assignment disruption or unnecessary membership churn when correctly configured for the selected protocol.

VISUAL MEMORY MAP

Read left to right

Subscribed group → Partition assignment → Progress checkpoint

WORKED EXAMPLE

```
group.instance.id=orders-pod-unique-id
```

WHY IT WORKS

Static membership requires a stable unique identity per group member. Cooperative strategies incrementally transfer ownership rather than treating all partitions as immediately revoked. They reduce disruption but do not remove timeouts or scaling tradeoffs.

COMMON INTERVIEW TRAP

Duplicate static identities can fence a member.

FOLLOW-UP QUESTION

How would you assign stable identities in Kubernetes?

Use StatefulSet ordinal or another stable per-pod identity combined with a stable group name. Ensure every live consumer has a unique `group.instance.id` and plan identities for scaling.

07. Offsets and acknowledgment

Committed offsets are restart checkpoints, not business transaction commits.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

What does a committed offset represent?

THE SIMPLE ANSWER

It records the next position the group should resume from for a partition.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Processed offset 41 -> commit next offset 42

WHY IT WORKS

The consumer's current position can be ahead of its committed position after polling. Commit is a progress checkpoint, not deletion of the record. A restart resumes from committed progress if the offset remains valid.

COMMON INTERVIEW TRAP

Committing 41 after processing 41 can replay that record.

FOLLOW-UP QUESTION

How would you inspect current and committed positions?

Use AdminClient or `kafka-consumer-groups --describe` for committed offsets; use KafkaConsumer.position for its current next fetch position. Compare with log-end offsets separately.

What does auto.offset.reset control?

THE SIMPLE ANSWER

It chooses behavior when the group has no valid committed offset, not on every restart.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.consumer.auto-offset-reset=earliest
```

WHY IT WORKS

With a valid commit, the consumer resumes there even if earliest is configured. A fresh group can read retained history. If retention removed an offset, the reset policy becomes important. Use controlled resets for intentional replay.

COMMON INTERVIEW TRAP

earliest does not force an existing group to reread all records.

FOLLOW-UP QUESTION

How do you replay without affecting production progress?

Run a new isolated group (or shadow application) with an explicit starting offset and write to a separate projection. Leave production group IDs and committed positions unchanged.

Why disable Kafka auto-commit for container-managed processing?

THE SIMPLE ANSWER

It lets Spring's acknowledgment mode coordinate progress with listener outcomes instead of periodic client commits.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.consumer.enable-auto-commit=false
   spring.kafka.listener.ack-mode=record
```

WHY IT WORKS

For a record listener, RECORD commits after successful record handling under the container's commit configuration. Default container modes and transactional behavior differ. Read the effective configuration rather than assuming manual acknowledgment is always required.

COMMON INTERVIEW TRAP

Disabling auto-commit alone does not stop the Spring container from committing.

FOLLOW-UP QUESTION

What controls commits after auto-commit is disabled?

Spring Kafka listener container ack mode, error handler and transactional configuration govern commits. For example RECORD commits after successful listener handling under its configured rules.

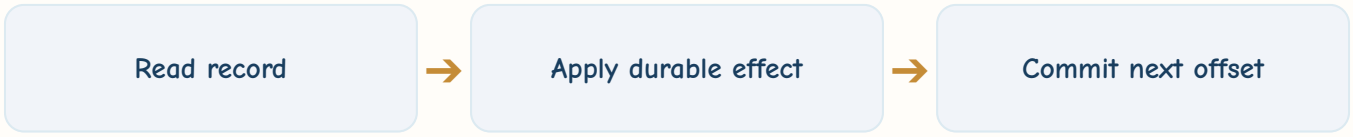
How do RECORD, BATCH and manual acknowledgment differ?

THE SIMPLE ANSWER

They choose record-level, poll-batch-level or application-signaled progress boundaries for nontransactional container processing.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

MANUAL: acknowledge queues progress by container rules
MANUAL_IMMEDIATE: immediate on consumer thread

WHY IT WORKS

BATCH commits after the records from a poll are handled. MANUAL is not a promise of an immediate commit. Calling acknowledgment off the consumer thread can alter timing. Error handling and transactions further affect commit behavior.

COMMON INTERVIEW TRAP

Acknowledging does not make the database and Kafka atomic.

FOLLOW-UP QUESTION

Why can a successful database write still be replayed?

After committing the DB change, the process can crash before Kafka stores the next offset. On restart the same record is delivered again; make the database effect idempotent.

How do at-most-once and at-least-once processing differ?

THE SIMPLE ANSWER

Committing before the required effect risks loss; committing afterward risks duplicates during a crash window.

VISUAL MEMORY MAP

Read left to right

commit → charge → crash before charge: lost work

WORKED EXAMPLE

```
commit -> charge -> crash before charge: lost work
  charge -> crash -> no commit: possible duplicate charge
```

WHY IT WORKS

At-least-once with an idempotent business effect is often the practical choice. Define the exact durable effect and checkpoint. Manual commits do not eliminate the fundamental crash interval.

COMMON INTERVIEW TRAP

Treating duplicates as impossible is a design defect.

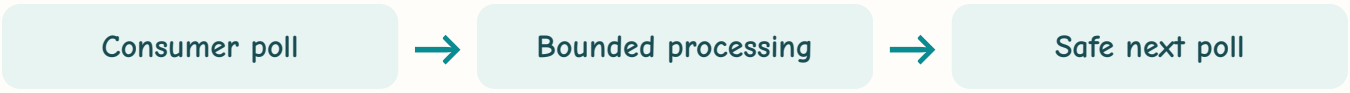
FOLLOW-UP QUESTION

How would a unique eventId make a charge safe?

Put a unique `(paymentId, effectType)` constraint or `(consumerId,eventId)` inbox entry in the same database transaction as the charge ledger update. Replays then find the previous result.

08. Poll loop and processing pressure

Slow processing and unbounded work cause lag and rebalance risk.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

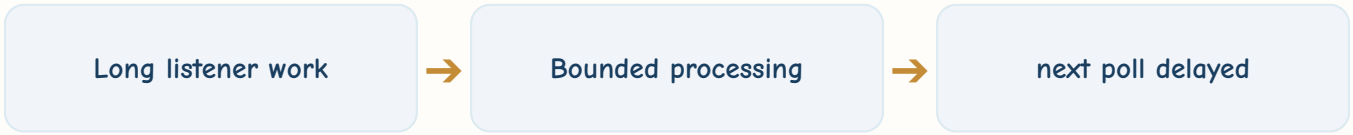
How do heartbeat and max.poll.interval.ms differ?

THE SIMPLE ANSWER

Heartbeats maintain liveness; the poll interval limit checks whether the consumer continues making polling progress.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Long listener work -> next poll delayed
Heartbeats alone do not justify unlimited processing time

WHY IT WORKS

Classic and newer consumer protocols differ in which timing settings are client- or broker-controlled. For the classic baseline, budget the worst poll batch plus retry time below the interval. Identify the actual group protocol first.

COMMON INTERVIEW TRAP

Increasing heartbeat frequency does not fix an oversized poll workload.

FOLLOW-UP QUESTION

Which timing is exceeded by a slow batch?

For classic groups, a slow batch risks exceeding `max.poll.interval.ms`, even if heartbeats occur. Choose batch size and processing budget within the actual protocol's polling rules.

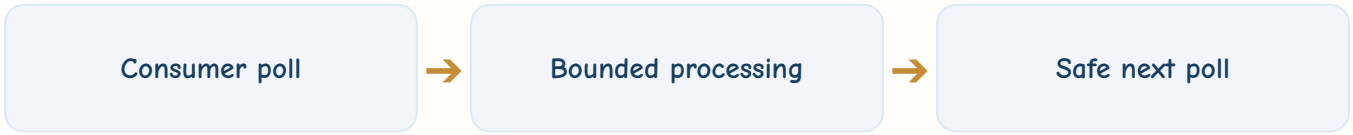
What does max.poll.records limit?

THE SIMPLE ANSWER

It limits the records returned per poll, not the entire broker fetch byte size.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
max.poll.records=100
  100 records * 50 ms serial work = about 5 seconds
```

WHY IT WORKS

Fetched data may be cached by the client and returned across polls. Tune returned batch size against processing time, heap and downstream capacity. A few exceptionally expensive records can dominate the budget.

COMMON INTERVIEW TRAP

Lowering max.poll.records does not strictly cap all fetched bytes.

FOLLOW-UP QUESTION

Which settings and metrics describe fetch memory?

Consider `fetch.max.bytes`, `max.partition.fetch.bytes`, `fetch.min.bytes`, `fetch.max.wait.ms`, memory used for deserialization, and bytes per partition. Watch fetch size and heap/GC metrics.

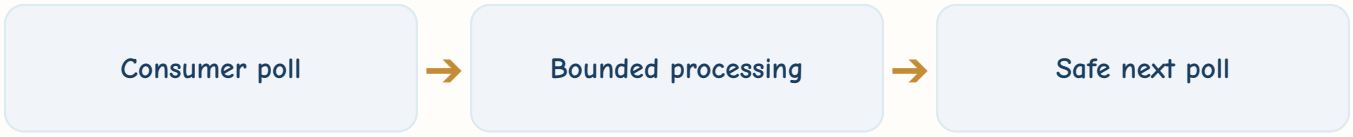
Can KafkaConsumer be shared across worker threads?

THE SIMPLE ANSWER

KafkaConsumer is not generally thread-safe; coordinate consumer operations on its owning thread.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Consumer thread: poll, assignment, commit
  Workers: bounded business tasks with explicit ordering

WHY IT WORKS

Delegating records to workers requires partition-aware completion tracking, revocation handling and safe commits. Committing beyond unfinished earlier work can skip it on restart. Prefer container concurrency when it meets the requirement.

COMMON INTERVIEW TRAP

Offloading and immediately returning marks work complete too early.

FOLLOW-UP QUESTION

How do you track the highest contiguous completed offset?

For each partition track completed offsets and hold gaps in an ordered set. Commit only one past the highest contiguous completed offset; never skip an unfinished earlier record.

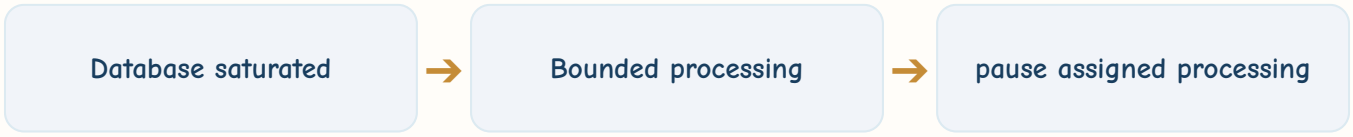
When should a listener pause consumption?

THE SIMPLE ANSWER

Pause when temporary downstream saturation requires bounded intake while retaining controlled polling and membership.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Database saturated -> pause assigned processing
  Capacity restored -> resume
```

WHY IT WORKS

Use container-supported pause/resume and observe actual pause state. Maintain a finite queue and a recovery threshold to avoid oscillation. Pausing a consumer does not pause producers or stop topic retention.

COMMON INTERVIEW TRAP

An unbounded in-memory queue only moves lag out of Kafka.

FOLLOW-UP QUESTION

What would trigger pause and resume?

Pause when queue utilization or DB latency exceeds a high threshold; resume only after load falls below a lower threshold. Monitor paused duration and retained Kafka backlog.

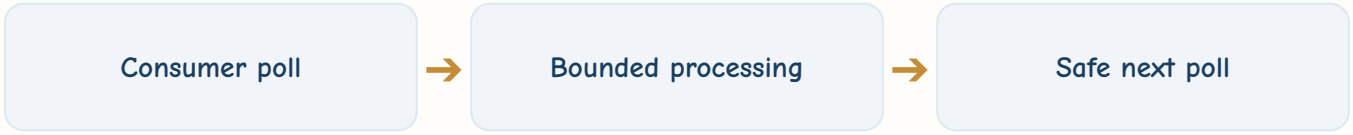
How should you shut down a consumer safely?

THE SIMPLE ANSWER

Stop new work, finish or abandon bounded in-flight tasks deliberately, and let the container close and commit according to its policy.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Termination budget > listener timeout + commit/close allowance

WHY IT WORKS

Long external calls need deadlines; otherwise the process may be killed mid-effect. Replayed work must remain idempotent. Coordinate readiness and termination grace with the container's stop behavior.

COMMON INTERVIEW TRAP

Graceful shutdown cannot guarantee no duplicates after every crash.

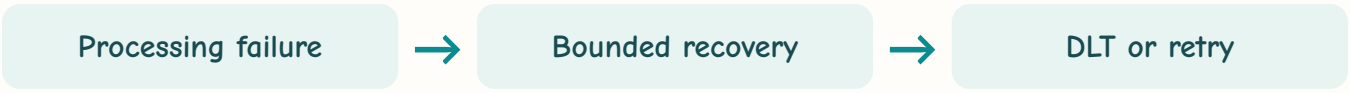
FOLLOW-UP QUESTION

What test would simulate forced termination?

Terminate a consumer process during a slow DB write, after the DB commit and before Kafka commit, then restart. Verify safe replay, no skipped durable work, and bounded shutdown.

09. Failures, retries and dead-letter topics

Retry policies must be bounded and must not silently discard durable work.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

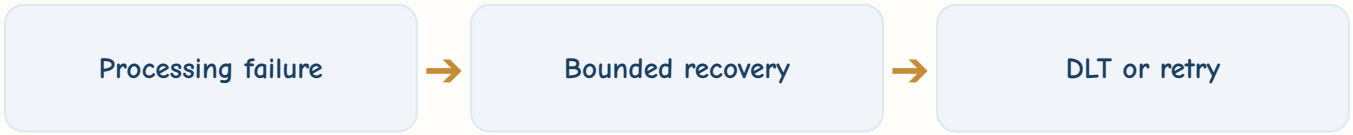
Why should a listener propagate processing failures?

THE SIMPLE ANSWER

The container must observe failure to apply retry, rollback or recovery rules.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
try { charge(event); }
   catch (Exception e) { throw e; }
```

WHY IT WORKS

Logging and returning normally can advance progress after failed work. Catch only when you complete a documented alternative outcome durably. Exception classification should separate transient infrastructure errors from permanently invalid events.

COMMON INTERVIEW TRAP

A log line is not a recovery action.

FOLLOW-UP QUESTION

Which failures should never retry?

Do not blindly retry invalid schemas, unauthorized actions or permanent business rejections. Quarantine poison records with evidence; use retries for transient failures under a bounded policy.

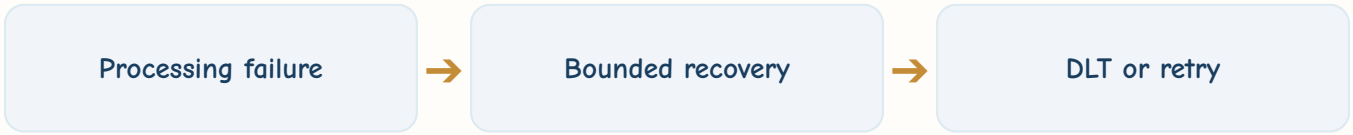
What does DefaultErrorHandler do?

THE SIMPLE ANSWER

It applies configured retry/backoff and recovery behavior for listener failures in nontransactional containers.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 2L));
```

WHY IT WORKS

Two backoff retries means an initial attempt plus two retries. Seek/replay behavior, exception classification and acknowledgment settings control what happens afterward. Configure a durable recoverer instead of assuming every exhausted failure is retained safely.

COMMON INTERVIEW TRAP

Exhaustion without a deliberate recovery policy can skip business work.

FOLLOW-UP QUESTION

How do you verify recovery failure does not advance progress?

Force the recoverer/DLT producer to fail (for example revoke write permission), restart the listener, and verify that source progress does not pass the record without the configured durable recovery guarantee.

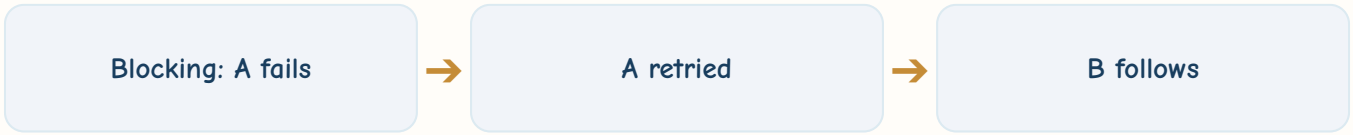
How do blocking and nonblocking retries differ?

THE SIMPLE ANSWER

Blocking retries delay source processing; retry topics move failed records to later processing on other topics.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Blocking: A fails -> A retried -> B follows
Retry topic: A deferred -> B may run before A

WHY IT WORKS

Nonblocking retries improve source throughput but can break original ordering. Spring Kafka 3.3 retry-topic support is for record listeners and cannot combine with container transactions. Choose the strategy from business order and atomicity needs.

COMMON INTERVIEW TRAP

Do not attach @RetryableTopic to a transactional or batch design blindly.

FOLLOW-UP QUESTION

Which strategy fits sequential account updates?

For sequential account updates, blocking retries or a partition-preserving queue usually fit better. Retry topics can let a later account event finish before the earlier one.

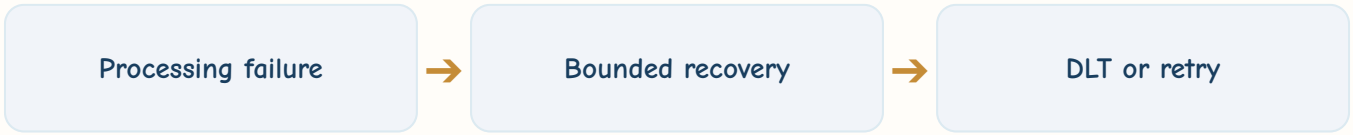
What belongs in a dead-letter topic?

THE SIMPLE ANSWER

An unprocessable event plus enough safe metadata to investigate, repair and replay it.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

DLT: original topic, partition, offset, eventId,
schema version, failure classification

WHY IT WORKS

A DLT is a recovery workflow, not a trash can. Define alerts, ownership, access control, retention and replay authorization. Verify publication success before treating a source record as recovered. Preserve original key semantics where needed.

COMMON INTERVIEW TRAP

Sensitive exception messages can leak data into DLT headers.

FOLLOW-UP QUESTION

Who approves and audits a replay?

An accountable operations or domain owner should authorize replay after root-cause repair, with approval records, event IDs, deduplication policy, target group and audit trail.

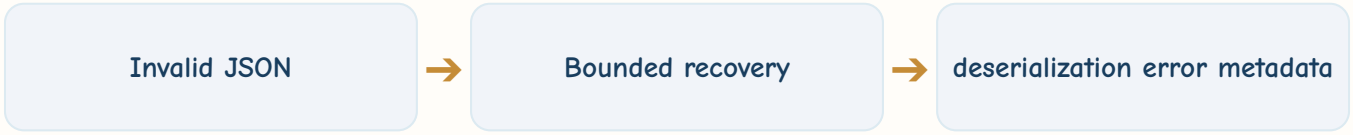
How do you handle deserialization failures?

THE SIMPLE ANSWER

Handle failures before normal listener conversion, for example with ErrorHandlingDeserializer and a compatible error/recovery setup.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Invalid JSON -> deserialization error metadata
  Error handler -> quarantine original bytes
```

WHY IT WORKS

The listener may never receive the intended object. Recovery producers must be able to serialize raw byte payloads when required, as well as normal values. Test invalid bytes and DLT publication failures.

COMMON INTERVIEW TRAP

A try/catch inside the POJO listener cannot catch every deserializer error.

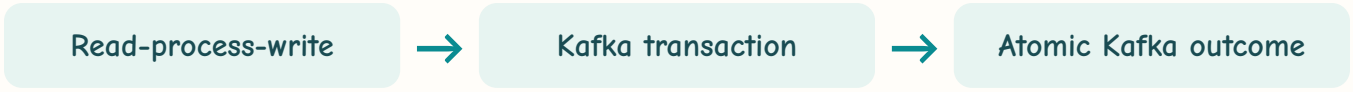
FOLLOW-UP QUESTION

Which serializer can publish the failed raw payload?

Spring Kafka can route raw failed bytes through `ErrorHandlingDeserializer` headers and use `DeadLetterPublishingRecoverer` with a byte-array serializer for the failed value where configured.

10. Kafka transactions and exactly-once boundaries

Kafka transactions coordinate Kafka outputs and offsets, not arbitrary external effects.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

What is a Kafka transaction?

THE SIMPLE ANSWER

It atomically commits or aborts a set of Kafka writes and, when included, consumed group offsets.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

begin -> publish output -> include input offsets -> commit

WHY IT WORKS

Read-process-write pipelines can commit outputs and progress together. Consumers must use the appropriate isolation to avoid observing aborted writes. The transaction does not include an arbitrary database or external HTTP service.

COMMON INTERVIEW TRAP

Atomic Kafka writes are not a global distributed transaction.

FOLLOW-UP QUESTION

What happens to an email sent before rollback?

The email provider is outside the Kafka transaction: the email may already be sent even if Kafka rolls back. Use provider-level idempotency or a durable delivery ledger.

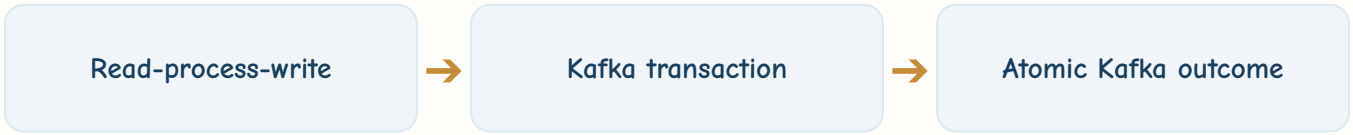
What does read_committed isolation do?

THE SIMPLE ANSWER

It returns committed transactional records and nontransactional records while excluding aborted transactional records.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.consumer.properties[isolation.level]=read_committed
```

WHY IT WORKS

Open transactions can hold back visibility beyond the last stable offset. Aborted records still occupy log positions, so visible offsets can have gaps. Read-uncommitted consumers do not get the same transactional visibility guarantee.

COMMON INTERVIEW TRAP

read_committed does not validate business semantics.

FOLLOW-UP QUESTION

Why can visible consumer lag grow during a long transaction?

A read_committed consumer waits past open transactions and hides aborted data. Its visible end offset can lag the physical log while a long-running transaction remains incomplete.

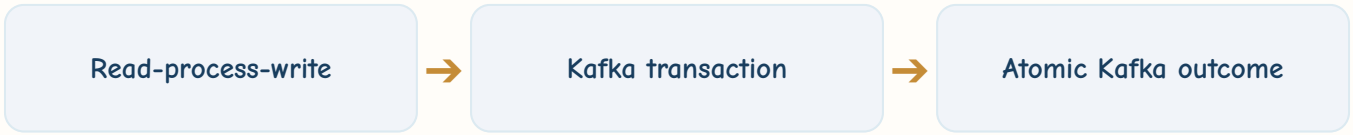
How does Spring configure a transactional listener pipeline?

THE SIMPLE ANSWER

Boot can configure a Kafka transaction manager from a producer transaction-id prefix and wire it into the default listener container.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.producer.transaction-id-prefix=${INSTANCE_ID}-
```

WHY IT WORKS

Use a unique prefix per application instance. With container transactions, listener output and input offsets can participate in one Kafka transaction. Spring Kafka 3.x uses EOS V2. Validate the custom factory wiring if you replace Boot defaults.

COMMON INTERVIEW TRAP

A shared transactional identity across active instances can cause fencing.

FOLLOW-UP QUESTION

How do you verify the listener is actually transactional?

Test that the listener container uses a transactional Kafka producer/transaction manager and that consumed offsets join its transaction. Inject an exception and verify output and offsets are not committed.

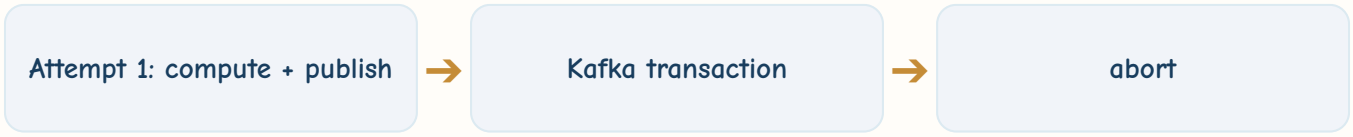
What does exactly-once mean for a Kafka read-process-write pipeline?

THE SIMPLE ANSWER

The committed Kafka outcome can be exactly once even though processing code is invoked again after rollback.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Attempt 1: compute + publish -> abort
Attempt 2: compute + publish + offsets -> commit

WHY IT WORKS

Spring documents the read/process steps as at-least-once; the committed sequence is the EOS boundary. External effects can repeat, so a database write or payment request still needs its own correctness mechanism.

COMMON INTERVIEW TRAP

Never claim the listener method executes only once.

FOLLOW-UP QUESTION

How would you explain EOS in one interview sentence?

Exactly-once for Kafka read-process-write means produced Kafka records and consumed offsets commit atomically in one Kafka transaction; external databases and emails remain outside that boundary.

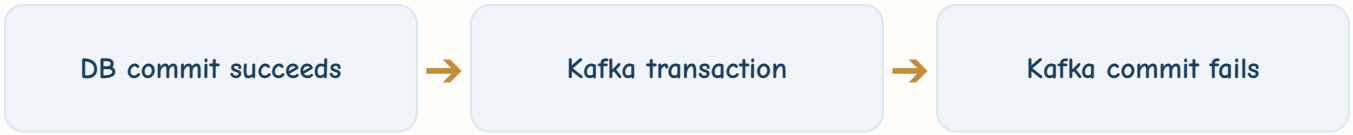
What happens when database and Kafka transactions are synchronized?

THE SIMPLE ANSWER

They still have separate commit boundaries, so a later commit can fail after the earlier system committed.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

DB commit succeeds -> Kafka commit fails
Required response: recovery / reconciliation

WHY IT WORKS

Spring synchronization can coordinate order but is not a general two-phase atomic transaction across both resources. Avoid deprecated chained transaction manager designs as a magic solution. Use an outbox or explicitly accepted compensation boundary.

COMMON INTERVIEW TRAP

@Transactional alone cannot guarantee atomic database-plus-Kafka publication.

FOLLOW-UP QUESTION

How would an outbox close the missing-publication gap?

Write business state and an outbox event in one local DB transaction. A relay retries publication after crashes; consumers deduplicate the stable event ID, closing the lost-intent window.

11. Idempotency, outbox and business correctness

The database is where many business invariants and event deduplication must be enforced.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

How do you build an idempotent consumer?

THE SIMPLE ANSWER

Atomically deduplicate an event and apply its business effect in the same durable transaction.

VISUAL MEMORY MAP

Read left to right

Business DB change → Durable event identity → Replay-safe effect

CODE / WORKED EXAMPLE

```
BEGIN;
   INSERT processed_event(consumer_name,event_id) VALUES (...);
   UPDATE account SET balance=balance+...;
   COMMIT;
```

WHY IT WORKS

A unique constraint enforces the deduplication key under concurrent deliveries. Duplicate detection must be handled with the database's transaction semantics in mind; do not continue in a transaction marked rollback-only. Commit offsets after success.

COMMON INTERVIEW TRAP

A separate check-then-insert can race.

FOLLOW-UP QUESTION

Why include consumer identity in the deduplication key?

Different consumers can legitimately apply the same event for different purposes. A uniqueness key such as `(consumerName,eventId)` prevents duplicates per effect without blocking other consumers.

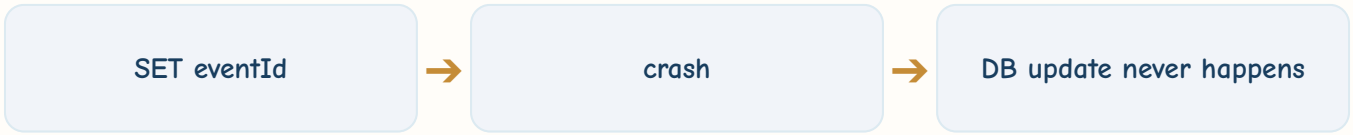
Why is a Redis deduplication flag insufficient for a database effect?

THE SIMPLE ANSWER

A separate flag and database update have a failure gap unless a deliberate cross-system protocol closes it.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
SET eventId -> crash -> DB update never happens
  DB update -> crash -> SET never happens
```

WHY IT WORKS

TTL expiry can also allow old replays through. Prefer a database invariant for database effects, or an idempotency key accepted by the external provider. Redis may optimize reads but should not silently define stronger guarantees.

COMMON INTERVIEW TRAP

A cache hit is not proof a durable effect committed.

FOLLOW-UP QUESTION

What should the payment provider's idempotency contract say?

The provider should accept a stable scoped idempotency key, store the canonical request fingerprint and final charge result, return the original result on a replay, and reject key reuse with different input.

What is the transactional outbox pattern?

THE SIMPLE ANSWER

Save business data and an event row in one database transaction, then publish the row asynchronously.

VISUAL MEMORY MAP

Read left to right

Business DB change → Durable event identity → Replay-safe effect

CODE / WORKED EXAMPLE

```
BEGIN: insert order + insert outbox event; COMMIT
   Publisher: read committed outbox -> Kafka -> mark delivered
```

WHY IT WORKS

If publication succeeds but marking fails, the event can be published again. Therefore the consumer still deduplicates. The outbox provides durable publication intent, not inherently exactly-once transport.

COMMON INTERVIEW TRAP

Marking delivered before Kafka confirmation can lose the event.

FOLLOW-UP QUESTION

How would you coordinate multiple outbox publishers?

Use database row claims such as `FOR UPDATE SKIP LOCKED`, leases or version-checked updates with small batches. Republish safely using stable IDs because a crash may occur after send.

How do polling and CDC outbox publishing differ?

THE SIMPLE ANSWER

Polling queries pending rows; CDC reads committed database changes from a change log through a connector.

VISUAL MEMORY MAP

Read left to right

Business DB change → Durable event identity → Replay-safe effect

WORKED EXAMPLE

Polling: SELECT pending ORDER BY ...
CDC: database log -> connector -> Kafka

WHY IT WORKS

Polling needs claiming, indexes and cleanup. CDC needs connector lifecycle, log retention, snapshots and routing configuration. Both require stable event IDs, schema contracts and monitoring of publication delay.

COMMON INTERVIEW TRAP

CDC does not remove consumer duplicate handling.

FOLLOW-UP QUESTION

What happens when the database log expires during connector downtime?

If the connector stays down past log retention, required database changes may be unavailable to CDC. Restore from backup/snapshot and rebuild consistently, or use a durable snapshot-and-resume strategy.

How do you prevent stale or reordered events overwriting new state?

THE SIMPLE ANSWER

Enforce aggregate version or another explicit monotonic invariant at the durable projection boundary.

VISUAL MEMORY MAP

Read left to right

Business DB change → Durable event identity → Replay-safe effect

CODE / WORKED EXAMPLE

```
UPDATE projection SET version=8, status='SHIPPED'
   WHERE id=42 AND version < 8;
```

WHY IT WORKS

For a full-state projection, a newer version can replace an older state. For incremental deltas, missing versions may require buffering or repair rather than skipping. Choose behavior from the event meaning.

COMMON INTERVIEW TRAP

Discarding old deltas can lose required arithmetic updates.

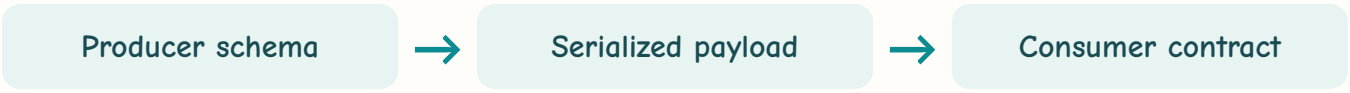
FOLLOW-UP QUESTION

How would you reconcile a missing version?

Detect a version gap (received 12 after 10), defer the newer update, fetch the missing 11 from the owner or replay source, and reconcile with a snapshot if the event is no longer retained.

12. Serialization and schema evolution

Choose a versioned event contract; payload bytes must be interpreted safely.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

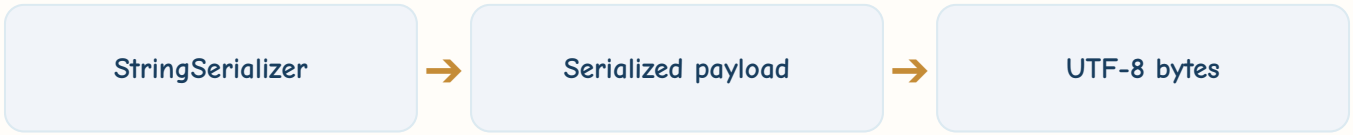
What do serializer and deserializer do?

THE SIMPLE ANSWER

They convert application keys/values to bytes and bytes back to usable data.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
StringSerializer -> UTF-8 bytes
   StringDeserializer -> String
```

WHY IT WORKS

Both ends need an agreed format. Spring message conversion is a separate layer that can convert already-deserialized content for a method parameter. Explicit contracts simplify consumers written in other languages.

COMMON INTERVIEW TRAP

A matching Java generic type does not prove matching wire format.

FOLLOW-UP QUESTION

How would you debug a serializer mismatch?

Inspect record topic/partition/offset, declared serializer format, headers and content-type/schema ID. Check that producer and consumer agree on key/value classes, encodings and trusted types.

How should a business event envelope look?

THE SIMPLE ANSWER

Include stable identity, type, version, occurrence time and the domain payload needed by consumers.

VISUAL MEMORY MAP

Read left to right

Producer schema → Serialized payload → Consumer contract

CODE / WORKED EXAMPLE

```
{"eventId":"e-42","type":"OrderPlaced","version":1,
    "occurredAt":"2026-10-08T06:00:00Z","orderId":"42"}
```

WHY IT WORKS

Avoid embedding secrets or a complete mutable database entity. Separate event identity from correlation and aggregate identity. Define required fields, nullable values, number precision and time zone semantics.

COMMON INTERVIEW TRAP

An eventId generated anew for every retry defeats deduplication.

FOLLOW-UP QUESTION

Which fields remain unchanged during replay?

Keep original eventId, aggregateId, logical version and the original occurrence timestamp unchanged. Add separate replay metadata or attempt count rather than silently minting a new business event.

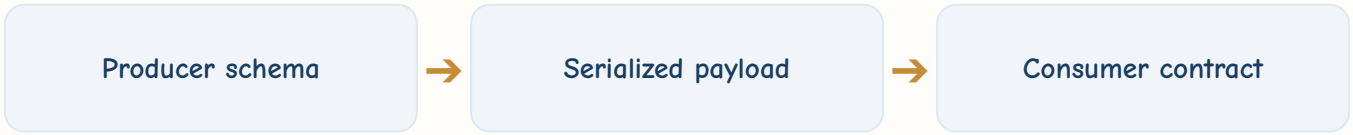
How do you evolve JSON events safely?

THE SIMPLE ANSWER

Prefer compatible additive changes and tolerant readers, while validating fields required for business correctness.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

v1: orderId, amount
v2: orderId, amount, optional currency defaulted by contract

WHY IT WORKS

An optional field is safe only if its default has an agreed meaning. Renaming, deleting or changing units can break consumers even when JSON parses. Test old readers/new writers and new readers/old records.

COMMON INTERVIEW TRAP

Parse success does not imply semantic compatibility.

FOLLOW-UP QUESTION

How would you change amount from rupees to paise?

Add a new `amountMinorUnits` field with clear currency and transition semantics, backfill or dual-publish during migration, test old readers, and retire the old rupees field only after compatibility checks.

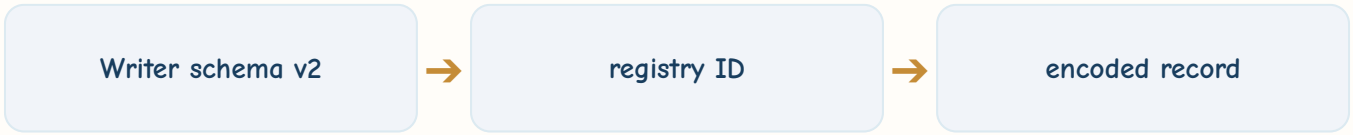
What does a schema registry provide?

THE SIMPLE ANSWER

It manages schemas and compatibility checks for configured serialization ecosystems such as Avro or Protobuf.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Writer schema v2 -> registry ID -> encoded record
Reader resolves schema and applies compatibility rules

WHY IT WORKS

A schema registry is an additional component, not a built-in guarantee of every Kafka cluster. Subject naming, compatibility mode and deployment sequence matter. It catches structural changes but not every business meaning change.

COMMON INTERVIEW TRAP

A structurally compatible enum or unit change can still break business logic.

FOLLOW-UP QUESTION

Which compatibility mode suits long historical replay?

For long replay with multiple historical schemas, use compatibility rules that support all versions a new reader will encounter (often a transitive backward-compatible policy), and verify actual schemas.

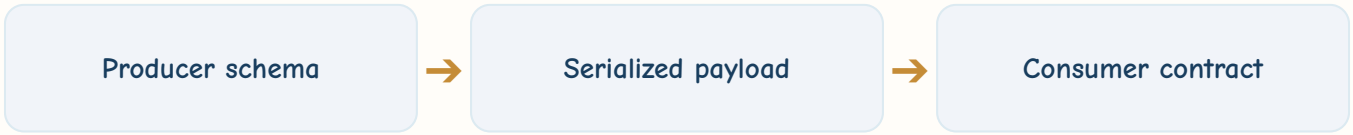
Why restrict trusted JSON types?

THE SIMPLE ANSWER

Type metadata from incoming messages must not authorize arbitrary class deserialization.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

JsonDeserializer configured with a specific target type
Trusted package list kept narrow

WHY IT WORKS

Spring Kafka JSON settings and class names differ across major versions. Prefer explicit type mapping or a fixed target type where appropriate. Treat an event source as untrusted even when the transport authenticates it.

COMMON INTERVIEW TRAP

trusted.packages=* is a broad trust decision.

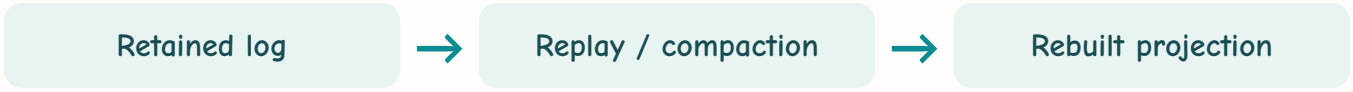
FOLLOW-UP QUESTION

How would you support multiple event types safely?

Use a documented type discriminator and explicit allowlist of event classes or type mappings; reject unrecognized types. Never allow untrusted data to pick arbitrary Java classes.

13. Retention, compaction and replay

Retention limits the available replay history; compaction changes the visible key history.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

How do delete retention and compaction differ?

THE SIMPLE ANSWER

Delete retention removes old segments by policy; compaction eventually keeps the latest value per key.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Delete: event history window
Compact: latest known state for each key

WHY IT WORKS

Compaction is asynchronous and can coexist with deletion. Older values may remain until cleaning occurs. A compacted topic is useful for rebuilding keyed state, but cannot be assumed to preserve every historical event.

COMMON INTERVIEW TRAP

Compaction is not immediate deduplication.

FOLLOW-UP QUESTION

Which policy fits an audit ledger?

An audit ledger normally needs a durable append-only history and a retention plan suitable for compliance. Pure compaction would discard historical changes and is usually inappropriate.

What is a tombstone?

THE SIMPLE ANSWER

It is a record with a non-null key and null value that represents deletion in a compacted topic.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

key=customer-42, value=null

WHY IT WORKS

Consumers rebuilding state must apply the deletion. Tombstone retention and consumer downtime affect whether a stale local entry can be removed reliably. An empty JSON object or the string "null" is not the same wire value.

COMMON INTERVIEW TRAP

A null key cannot express a specific keyed deletion.

FOLLOW-UP QUESTION

How would a long-offline consumer miss a tombstone?

Compaction and retention can eventually remove older tombstones. A consumer offline too long may miss deletion evidence and need a rebuilt snapshot or an external deletion ledger.

How do you replay safely?

THE SIMPLE ANSWER

Use a controlled group or offset reset, verify retention coverage, and isolate or deduplicate external side effects.

VISUAL MEMORY MAP

Read left to right

Retained log → Replay / compaction → Rebuilt projection

WORKED EXAMPLE

New group: billing-rebuild-2026
Target: fresh projection table, no real payment calls

WHY IT WORKS

Record replay scope, schema versions, rate limits and expected result counts. An existing live group reset can reapply business effects and disrupt consumers. Rebuilding projections is often safer than repeating original commands.

COMMON INTERVIEW TRAP

earliest reads only currently retained data.

FOLLOW-UP QUESTION

How would you compare the rebuilt and live projection?

Build a shadow projection from a known snapshot plus replay offset, then compare business aggregates, row counts, hashes or versions with the live view before cutover.

How should dead-letter events be replayed?

THE SIMPLE ANSWER

Repair the cause, preserve identity and provenance, then republish through an authorized rate-limited workflow.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

DLT event e-42 -> repaired source event e-42
Replay audit: operator, reason, original position

WHY IT WORKS

Use the original eventId when replaying the same business event; a genuinely new corrective action needs a distinct identity. Preserve or restore aggregate order where required. Avoid endless source-to-DLT loops.

COMMON INTERVIEW TRAP

Blindly replaying every DLT record can repeat permanent failures.

FOLLOW-UP QUESTION

How do you identify a successful previous effect?

Use the stable eventId or business operation ID to query the durable inbox or target effect. If it already succeeded, mark replay as duplicate rather than repeating the payment/email.

What if a consumer is offline longer than retention?

THE SIMPLE ANSWER

It may lose access to unprocessed history and needs a snapshot, alternate archive or repair process.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
Committed next=100; earliest available=900
   Missing history: 100..899 may no longer be recoverable here
```

WHY IT WORKS

Alert on lag age relative to retention before the gap occurs. Retention bytes can also constrain available history. The reset policy controls resume behavior but cannot reconstruct removed records.

COMMON INTERVIEW TRAP

A large numeric retention setting does not guarantee unlimited history.

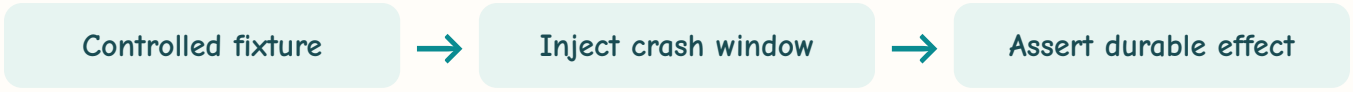
FOLLOW-UP QUESTION

Where would you source a recovery snapshot?

Load a trusted snapshot from the authoritative business database or archived event stream, record its cutover offset/version, then replay what remains. Do not assume Kafka retains history forever.

14. Testing Spring Boot Kafka applications

Test crash windows and durable effects, not only that a listener method was called.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

What belongs in a unit test?

THE SIMPLE ANSWER

Business mapping, validation, exception classification and idempotency decisions that do not require a broker.

VISUAL MEMORY MAP

Read left to right

Given invalid currency → Inject crash window → reject before publish

WORKED EXAMPLE

Given invalid currency -> reject before publish
Given duplicate event -> no second business effect

WHY IT WORKS

Mocking KafkaTemplate can test how application code handles a failed future, but it does not test partitioning, commits, rebalances or broker transactions. Keep a smaller set of real integration tests for those behaviors.

COMMON INTERVIEW TRAP

A mocked successful send does not demonstrate delivery.

FOLLOW-UP QUESTION

Which behavior needs a real broker?

Pure validation, mapping, partition-key choice and retry classification fit unit tests. Partition ownership, offsets, broker failures, transactions and DLT publication require a real Kafka integration test.

When should you use EmbeddedKafka or Testcontainers?

THE SIMPLE ANSWER

Use them for integration tests of actual broker interaction; choose based on fidelity and environment needs.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

EmbeddedKafka: fast in-process test fixture
Testcontainers: containerized broker with explicit image

WHY IT WORKS

Align the broker mode and version with the test objective. Isolate topics and groups, wait for assignment or signals and clean up resources. Broker integration is necessary for offset and transaction assertions.

COMMON INTERVIEW TRAP

A test fixture is not a production cluster reliability simulation.

FOLLOW-UP QUESTION

How would you test a three-broker durability policy?

Use a controlled multi-broker Kafka Testcontainers or dedicated test cluster with RF=3 and min ISR=2. Kill a leader/zone, test acknowledgments and recovery, and inspect ISR transitions.

Why are sleeps weak synchronization in Kafka tests?

THE SIMPLE ANSWER

They guess when asynchronous work finishes and make tests flaky or unnecessarily slow.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Await until processedEventCount == 1 within 10 seconds
    Then assert stored state and commit boundary
```

WHY IT WORKS

Use bounded polling, latches or observability hooks. A condition should assert the intended invariant, not merely that a listener was invoked. Choose timeouts generous enough for CI but fail with useful diagnostics.

COMMON INTERVIEW TRAP

Seeing a callback does not prove the database committed.

FOLLOW-UP QUESTION

What evidence would your failure message include?

Wait for explicit conditions using polling assertions and time bounds (e.g. Awaitility), not fixed sleeps. On failure report topic, partition, offset, consumed records, error and pending assertions.

How do you test duplicate handling meaningfully?

THE SIMPLE ANSWER

Deliver the same stable event identity more than once and verify one durable business effect.

VISUAL MEMORY MAP

Read left to right

Controlled fixture → Inject crash window → Assert durable effect

WORKED EXAMPLE

Publish eventId=e-42 twice
Assert balance incremented once and dedup row exists

WHY IT WORKS

Include concurrent delivery and restart after the effect but before progress commit. For database-backed deduplication, use the actual database constraint and transaction behavior rather than a mocked exists method.

COMMON INTERVIEW TRAP

Sequential mocked calls do not prove race safety.

FOLLOW-UP QUESTION

How would you force the crash window?

Commit the DB write, stop the process before Kafka offset commit, then restart with the same group. Verify a replay occurs and the unique inbox/ledger prevents a second business effect.

How do you test Kafka transactions?

THE SIMPLE ANSWER

Verify committed outputs are visible to read_committed consumers and aborted outputs are not, with input progress moving only on success.

VISUAL MEMORY MAP

Read left to right

Fail listener after output send → Inject crash window → transaction abort

WORKED EXAMPLE

Fail listener after output send -> transaction abort
Retry succeeds -> one committed output outcome

WHY IT WORKS

Use the production transaction manager and compatible broker fixture. Inspect offsets and output records, not only exception handling. Also test fencing or instance identity conflicts where relevant.

COMMON INTERVIEW TRAP

read_uncommitted output assertions weaken the test.

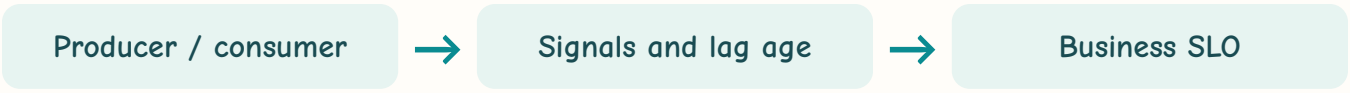
FOLLOW-UP QUESTION

What external side effect remains outside this test's guarantee?

An HTTP payment call or email delivery performed from the listener is not rolled back by a Kafka transaction. Test its own provider identity/recovery protocol separately.

15. Observability and performance

Client metrics must connect to business freshness and recovery objectives.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

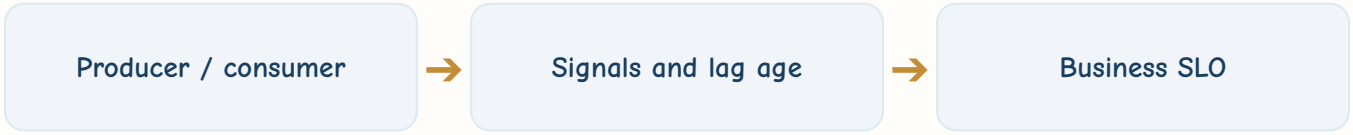
What is consumer lag?

THE SIMPLE ANSWER

It measures distance between a partition's end and the group's progress; its exact interpretation depends on the metric source and isolation.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
Partition end=1000; group committed next=800
   Approximate offset lag=200
```

WHY IT WORKS

Offset gaps and variable record cost mean lag is not an exact count of business tasks. Add age of oldest unprocessed event and processing rate. Inspect each partition rather than only group totals.

COMMON INTERVIEW TRAP

Zero commit lag does not prove every business effect succeeded.

FOLLOW-UP QUESTION

How can swallowed exceptions hide failure?

A listener can catch and ignore an exception, return normally and advance its offset. Lag may look healthy while the durable effect is missing; monitor business completion and error counters.

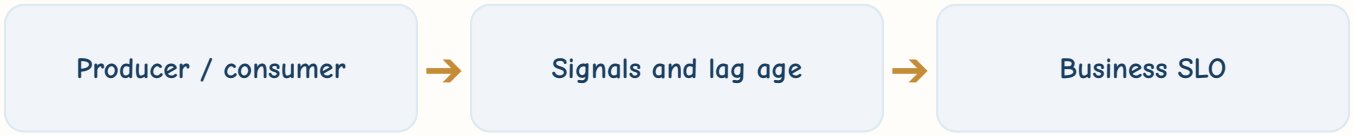
How do you diagnose increasing lag?

THE SIMPLE ANSWER

Compare arrival and completion rates, then identify whether the bottleneck is partition skew, processing, retries or infrastructure.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
Input=2000 records/s; completion=1500 records/s
   Backlog grows about 500 records/s for this workload
```

WHY IT WORKS

Check poll duration, rebalance counts, per-partition lag, external call latency, database saturation and broker fetch behavior. Scaling consumers helps only where partition ownership and downstream capacity permit.

COMMON INTERVIEW TRAP

Increasing concurrency can overload the database and worsen lag.

FOLLOW-UP QUESTION

What evidence would justify more partitions?

Add partitions only if multiple partitions are saturated and keys can be redistributed without violating ordering. Confirm CPU, records-per-partition, consumer parallelism and downstream headroom.

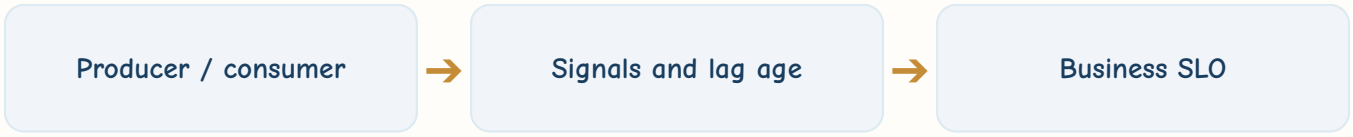
Which Kafka application metrics should be monitored?

THE SIMPLE ANSWER

Monitor publish errors and latency, consumer lag/age, processing failures, retries, DLT traffic and dependency saturation.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

SLO: 99% of OrderPlaced projected within 30 seconds
Alert: oldest pending event age > agreed threshold

WHY IT WORKS

Client metrics and Spring observations complement business outcome metrics. Bound labels such as topic and outcome; avoid unique eventIds as metric tags. Use eventId and correlationId in structured logs instead.

COMMON INTERVIEW TRAP

Consumer running is weaker evidence than business convergence.

FOLLOW-UP QUESTION

How do you keep metric cardinality bounded?

Use bounded labels such as topic, group, outcome and environment. Never label metrics with eventId, customerId or arbitrary payload fields; keep those in sampled traces/logs.

How should tracing work across Kafka?

THE SIMPLE ANSWER

Propagate trace context through approved headers and create spans that reflect asynchronous producer/consumer processing.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

HTTP span -> publish span -> consumer processing span
Logs: eventId, aggregateId, traceId, partition, offset

WHY IT WORKS

Spring Kafka observations must be enabled and configured for the selected setup. Preserve business identity separately from trace identity. Do not log complete sensitive payloads for debugging.

COMMON INTERVIEW TRAP

A traceId is not a business deduplication key.

FOLLOW-UP QUESTION

How would you diagnose a replay across different traces?

Propagate event correlation IDs and trace context in headers, and store stable event IDs in logs. Cross-reference replay attempts with the original event even when they belong to new traces.

How do you tune throughput responsibly?

THE SIMPLE ANSWER

Measure realistic record sizes, keys and downstream work, then change one limiting factor at a time.

VISUAL MEMORY MAP

Read left to right

Producer / consumer → Signals and lag age → Business SLO

WORKED EXAMPLE

Benchmark: p95 end-to-end latency, records/s, bytes/s, CPU, heap, broker load and error rate

WHY IT WORKS

Producer batching/compression and consumer concurrency/poll size affect different boundaries. Include failure and recovery tests, because a configuration that wins a healthy benchmark may produce unstable retry storms.

COMMON INTERVIEW TRAP

A throughput number without durability settings is incomplete.

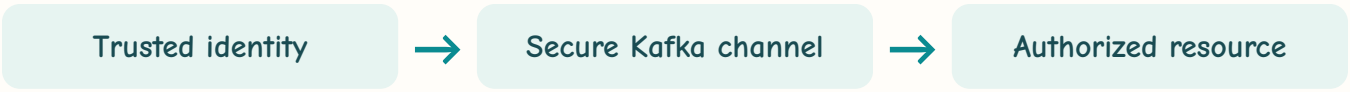
FOLLOW-UP QUESTION

Which SLO must not regress during tuning?

Preserve end-to-end processing freshness, durability and error-rate SLOs (for example p99 event-to-effect latency). Higher MB/s is not a win if lag age or DLT failures worsen.

16. Security and deployment

Encrypted links, authenticated clients and ACLs protect different boundaries.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

How do TLS, SASL and ACLs differ?

THE SIMPLE ANSWER

TLS protects transport and can authenticate certificates; SASL authenticates clients; ACLs authorize Kafka operations.

VISUAL MEMORY MAP

Read left to right

SASL_SSL → Secure Kafka channel → authenticated encrypted connection

WORKED EXAMPLE

SASL_SSL -> authenticated encrypted connection
ACL -> principal may read orders with approved group

WHY IT WORKS

Select mechanisms supported by the cluster and manage credentials securely. Permissions cover topic operations, consumer groups and transactional resources as needed. Transport security does not determine whether an event describes an authorized business action.

COMMON INTERVIEW TRAP

A valid client credential is not permission to every topic.

FOLLOW-UP QUESTION

Which ACLs does a transactional producer need?

Depending on the Kafka cluster and authorization setup, a transactional producer needs topic Write/Describe plus permission on its transactional ID (and possibly cluster/idempotent-write permission). Verify broker policy.

How should secrets be supplied to a Boot Kafka application?

THE SIMPLE ANSWER

Use a managed secret delivery mechanism and avoid committing credentials or exposing them through logs/endpoints.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
spring.kafka.properties.security.protocol=SASL_SSL
   Credential material injected from protected runtime configuration
```

WHY IT WORKS

Rotate credentials with a tested reconnect or deployment plan. Validate TLS trust and hostname verification instead of disabling them to fix connectivity. Keep DLTs and backups within the same data protection review.

COMMON INTERVIEW TRAP

A secret hidden in application.yml is still a secret in source control.

FOLLOW-UP QUESTION

How would you test credential rotation?

Rotate credentials in a staging cluster while traffic continues. Verify both old/new overlap, new connections, consumer rebalances, no lost records and eventual removal of old credentials.

What is bootstrap.servers versus advertised.listeners?

THE SIMPLE ANSWER

Bootstrap addresses locate the cluster; advertised addresses tell clients where to connect to individual brokers.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Bootstrap works -> metadata advertises broker:9092
Remote client cannot resolve broker -> subsequent calls fail

WHY IT WORKS

Docker, Kubernetes and external access often need different reachable listener addresses. Diagnose metadata routing, DNS, ports and TLS names. Replacing the bootstrap address alone cannot repair unreachable advertised endpoints.

COMMON INTERVIEW TRAP

A successful TCP connection to bootstrap is insufficient.

FOLLOW-UP QUESTION

How would you inspect broker metadata from the client network?

From the same network as the app, run Kafka CLI metadata commands or a small AdminClient against the bootstrap address. Confirm returned broker hostnames/ports resolve and are reachable from the pod.

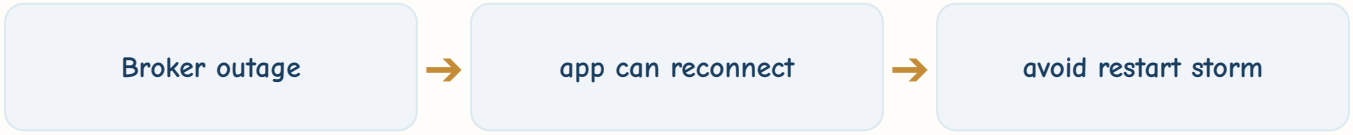
Should liveness fail whenever Kafka is unavailable?

THE SIMPLE ANSWER

Liveness should reflect whether the process can recover; readiness and health policy should represent whether it can fulfill its assigned role.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Broker outage -> app can reconnect -> avoid restart storm
HTTP write readiness may fail if acceptance cannot be durable

WHY IT WORKS

Producer-only APIs and background consumers have different admission needs. Define startup behavior, missing-topic behavior, reconnect strategy and dependency deadlines. An orchestrator restart cannot repair a remote cluster.

COMMON INTERVIEW TRAP

A healthy JVM does not prove publish capability.

FOLLOW-UP QUESTION

What degraded behavior is acceptable for your API?

Liveness should usually reflect a stuck application process, not a temporarily unreachable broker. Readiness may degrade if required event intake is unusable, while read-only APIs can stay available by policy.

How should rolling deployments affect event contracts?

THE SIMPLE ANSWER

Old and new producers/consumers must coexist safely during the rollout and rollback window.

VISUAL MEMORY MAP

Read left to right

Deploy tolerant consumers → Secure Kafka channel → deploy new writers

WORKED EXAMPLE

Deploy tolerant consumers -> deploy new writers
Rollback writers without invalidating stored records

WHY IT WORKS

Test schema compatibility, group membership changes and transaction identity uniqueness. Maintain capacity while instances drain. Database migrations and event versions need coordinated backwards-compatible sequencing.

COMMON INTERVIEW TRAP

Reusing a new event meaning under the old version breaks rollback.

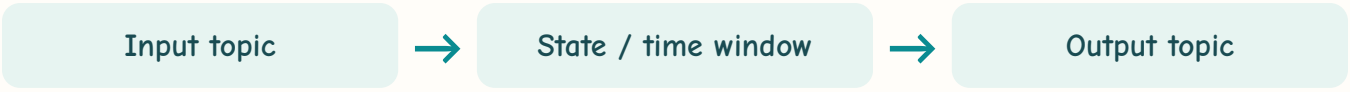
FOLLOW-UP QUESTION

How long must old event support remain?

Support old event schemas at least through the longest overlap of deployed consumers, retention and replay/recovery horizon. Prove backward compatibility before removing a field.

17. Streams and ecosystem choices

Use Streams when stateful event-time processing is useful; avoid unnecessary abstractions.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

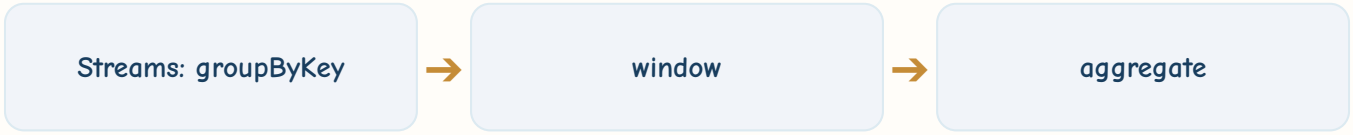
How does Kafka Streams differ from @KafkaListener?

THE SIMPLE ANSWER

Kafka Streams defines a processing topology with stateful operations; a listener is an application callback over consumed records.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Streams: groupByKey -> window -> aggregate -> output topic
  Listener: receive order -> call business service
```

WHY IT WORKS

Streams handles repartitioning, state stores and recovery within its topology model. A listener fits explicit workflow integration. Boot can support Streams configuration, but the programming model remains distinct.

COMMON INTERVIEW TRAP

Kafka Streams is not simply listener concurrency with a new name.

FOLLOW-UP QUESTION

When do you need a windowed aggregate?

Use Kafka Streams for stateful event-time joins, aggregates and windowing. A simple stateless consumer or DB write may be clearer with `@KafkaListener`.

What is a state store and changelog?

THE SIMPLE ANSWER

A state store holds local processing state; a changelog topic can support rebuilding that state after failure.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Store: customerId -> current total
  Changelog: durable state updates for recovery
```

WHY IT WORKS

Stateful processing introduces recovery time, disk needs and task ownership. Topic retention and topology naming affect restoration. Local query availability does not imply every instance has all keys.

COMMON INTERVIEW TRAP

Deleting changelog history can harm restoration.

FOLLOW-UP QUESTION

How would you estimate cold-start recovery time?

Estimate changelog bytes to restore, actual restore bandwidth and number of store partitions. Measure under cold cache and broker/network load; model restoration time plus rebalance delay.

What is a repartition topic?

THE SIMPLE ANSWER

It redistributes records by a new key so downstream keyed operations can process related records together.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Input keyed by orderId
  Select customerId -> repartition -> customer total
```

WHY IT WORKS

Changing an application key is a logical operation; stateful operations may need a physical shuffle. Repartitioning adds network, storage and topic-management cost. Plan topic names and topology upgrades intentionally.

COMMON INTERVIEW TRAP

Repartitioning is not free local computation.

FOLLOW-UP QUESTION

How would you identify unnecessary shuffles?

Inspect Kafka Streams topology descriptions, internal repartition topics and `groupBy`/key-changing operations. Preserve the key when possible so needless repartition topics are avoided.

How do event-time windows handle late records?

THE SIMPLE ANSWER

They group by event timestamps and use an explicit grace policy to decide how long late records can update results.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Window: 10:00-10:05
Late event timestamp=10:04 arrives later

WHY IT WORKS

Window type, timestamp extraction and grace determine acceptance. Stream time is driven by observed data, not simply the wall clock. Define whether results are provisional or final and what happens to dropped late records.

COMMON INTERVIEW TRAP

A five-minute window does not imply five minutes of wall-clock waiting.

FOLLOW-UP QUESTION

How would you explain grace to a business stakeholder?

Explain grace as extra time after the window end to accept late facts and correct results. After grace closes, the event needs a separate late-event policy rather than silently changing finalized totals.

How do Spring Cloud Stream and Spring Kafka differ?

THE SIMPLE ANSWER

Spring Cloud Stream provides a binder/function abstraction; Spring Kafka exposes Kafka-specific templates, listeners and container controls.

VISUAL MEMORY MAP

Read left to right



CODE / WORKED EXAMPLE

```
Cloud Stream: Consumer<OrderEvent> function + bindings
   Spring Kafka: @KafkaListener + container/error handler
```

WHY IT WORKS

Choose the abstraction from portability needs and required operational control. Binder configuration still relies on Kafka semantics underneath. Do not mix configuration property namespaces or assume all native options map identically.

COMMON INTERVIEW TRAP

An abstraction does not remove partitioning or commit decisions.

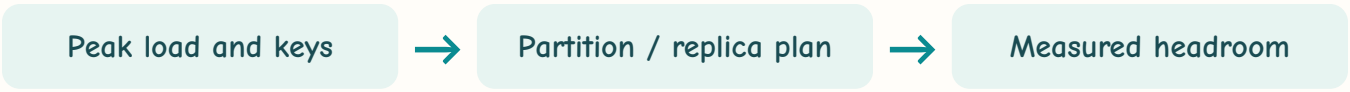
FOLLOW-UP QUESTION

Which setting needs native Kafka-specific control?

For native Kafka partition assignment, transaction configuration, `DefaultErrorHandler`, `@RetryableTopic`, low-level offset controls and producer tuning, Spring Kafka may expose the needed controls directly.

18. Architecture and sizing

Size by throughput, key skew, failures, retention and downstream capacity.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

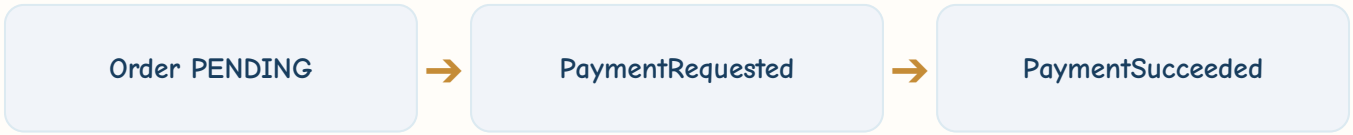
How would you design an order-to-payment workflow?

THE SIMPLE ANSWER

Commit order intent durably, publish through an outbox, process payment idempotently, and emit explicit outcomes.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Order PENDING -> PaymentRequested -> PaymentSucceeded
  Order CONFIRMED after permitted outcome transition
```

WHY IT WORKS

The payment provider should accept a stable idempotency key. Timeouts can mean unknown outcomes, so reconcile rather than issuing uncontrolled repeat charges. Apply valid state transitions using aggregate identity and version.

COMMON INTERVIEW TRAP

A Kafka acknowledgment is not payment confirmation.

FOLLOW-UP QUESTION

How do you recover a timed-out payment request?

Keep the original payment operation ID and set the order to PAYMENT_UNKNOWN after timeout. Query provider status or reconcile before retrying with the same key; do not charge with a new identity.

What is a saga in Kafka-based services?

THE SIMPLE ANSWER

It coordinates a multi-step workflow through local transactions and explicit compensating actions.

VISUAL MEMORY MAP

Read left to right

Reserve stock → charge payment → arrange shipment

WORKED EXAMPLE

Reserve stock -> charge payment -> arrange shipment
Failure -> release stock / refund under policy

WHY IT WORKS

Choreography uses emitted events; orchestration uses a coordinator. Neither creates a global ACID transaction. Compensations can fail and need durable retries, idempotency and human escalation.

COMMON INTERVIEW TRAP

A refund is not a literal rollback of history.

FOLLOW-UP QUESTION

Which action is irreversible in your workflow?

Shipping physical goods, sending irreversible messages or externally capturing funds may be difficult to undo. Choose the point of no return and design compensations, audit and manual recovery.

How do you estimate partition count?

THE SIMPLE ANSWER

Use measured per-partition throughput, desired consumer parallelism, skew and recovery headroom.

VISUAL MEMORY MAP

Read left to right

Peak load and keys → Partition / replica plan → Measured headroom

CODE / WORKED EXAMPLE

```
Required 12000 events/s
   Measured safe partition processing=2000 events/s
   Initial lower bound=6; add reviewed headroom
```

WHY IT WORKS

This is a sizing starting point, not a universal formula. Include payload bytes, brokers, replicas, open files, downstream concurrency and hot keys. Partition expansion can affect key order, so plan it early.

COMMON INTERVIEW TRAP

Average throughput hides burst and skew risk.

FOLLOW-UP QUESTION

What load test would validate the estimate?

Load-test representative key skew, average/peak message sizes, per-partition throughput, consumer processing p99, rebalances and downstream saturation. Check required headroom after a broker failure.

How do you estimate retained storage?

THE SIMPLE ANSWER

Start with incoming bytes times retention duration times replica count, then account for compression and operational overhead.

VISUAL MEMORY MAP

Read left to right

Peak load and keys → Partition / replica plan → Measured headroom

WORKED EXAMPLE

10 MB/s * 86400 s/day * 3 replicas

About 2.59 TB/day before compression and overhead

WHY IT WORKS

Use consistent units and measured stored compression ratio. Include indexes, segments, growth, rebalance movement and free-space headroom. Compaction sizing depends on key churn and cleaner progress.

COMMON INTERVIEW TRAP

Replication factor affects physical storage across brokers.

FOLLOW-UP QUESTION

What happens when one broker loses capacity?

With a broker down, remaining replicas handle leadership and replication load; spare disk and network capacity must exist. Alert on ISR loss and evaluate under-replication and retention backlogs.

How should a multi-tenant Kafka design enforce isolation?

THE SIMPLE ANSWER

Combine topic/key design, principal permissions and application tenant validation appropriate to the required boundary.

VISUAL MEMORY MAP

Read left to right

Peak load and keys → Partition / replica plan → Measured headroom

WORKED EXAMPLE

Event tenantId must match authorized operation
Projection writes constrained to tenant-owned data

WHY IT WORKS

Shared topics reduce topic count but rely more on application isolation. Separate topics and principals can strengthen operational boundaries at higher management cost. Validate DLT/replay paths as carefully as the happy path.

COMMON INTERVIEW TRAP

A tenantId field alone is not access control.

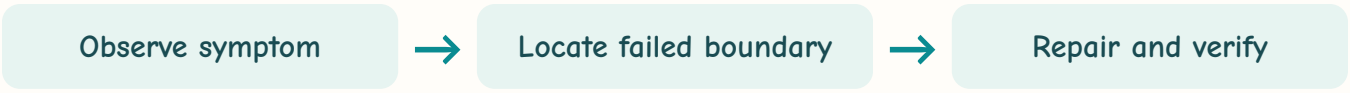
FOLLOW-UP QUESTION

Which tenants require dedicated infrastructure?

Regulated, noisy, high-volume or strict-isolation tenants may need separate topics, quotas, ACLs or clusters. Use risk and measurable resource contention rather than tenant count alone.

19. Production troubleshooting scenarios

Start at the first observable boundary and prove the failure with evidence.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

Events were published but one service sees nothing. Where do you start?

THE SIMPLE ANSWER

Check topic/cluster identity, group assignment, committed offsets, filtering and deserialization before blaming delivery.

VISUAL MEMORY MAP

Read left to right

Same group as another service? → Locate failed boundary → they divide work

WORKED EXAMPLE

Same group as another service? -> they divide work
Different group? -> independent subscription

WHY IT WORKS

Verify actual resolved bootstrap addresses and listener endpoint state. Inspect broker records with safe tooling and the group's progress. New groups with latest may not read older events.

COMMON INTERVIEW TRAP

Two services sharing a group do not each receive every record.

FOLLOW-UP QUESTION

How do you distinguish no records from skipped old history?

Check topic exists, producer confirms, subscription and group ID, partitions, committed offsets, reset policy and filters. A new group at latest can skip older retained records without any broker defect.

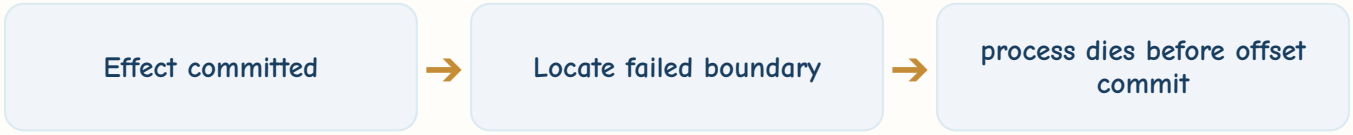
A listener runs twice for one event. Is Kafka broken?

THE SIMPLE ANSWER

Redelivery is expected after failure or a crash before durable progress commit; application republishing can also duplicate the event.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Effect committed -> process dies before offset commit
  New owner receives same source position
```

WHY IT WORKS

Compare eventId and topic/partition/offset. Same source position suggests redelivery; different positions with the same eventId suggest republishing. Repair the idempotency invariant instead of merely suppressing logs.

COMMON INTERVIEW TRAP

Producer idempotence does not prevent this consumer crash window.

FOLLOW-UP QUESTION

Which database constraint protects the effect?

No. At-least-once processing, retries or crash after business commit before offset commit can produce a repeat. Protect with a unique `(consumer,eventId)` or `(tenant,operationId)` database constraint.

Lag grows only on one partition. What is likely?

THE SIMPLE ANSWER

A hot key, expensive records or partition-specific failure can bottleneck that partition.

VISUAL MEMORY MAP

Read left to right

Observe symptom → Locate failed boundary → Repair and verify

CODE / WORKED EXAMPLE

```
Partition 0 lag=50000; others near zero
   Top key=one large tenant
```

WHY IT WORKS

Measure key distribution and dependency latency. More consumers cannot share that same partition within the group. Redesign key granularity only if ordering permits; otherwise optimize the serial path or isolate the hot aggregate.

COMMON INTERVIEW TRAP

Randomizing keys can destroy required entity order.

FOLLOW-UP QUESTION

What invariant constrains the redesign?

A single hot key or unusually slow records may dominate one partition. Preserve its required per-account ordering while adjusting aggregate design, processing speed or key strategy.

The app connects locally but fails in Kubernetes. What do you inspect?

THE SIMPLE ANSWER

Inspect advertised broker addresses, DNS, listener ports, TLS names, network policies and credentials from the pod network.

VISUAL MEMORY MAP

Read left to right

Metadata returns localhost:9092 → Locate failed boundary → points to the pod itself

WORKED EXAMPLE

Metadata returns localhost:9092 -> points to the pod itself

WHY IT WORKS

Compare bootstrap reachability with every advertised endpoint. Check error causes instead of increasing retries blindly. A TLS hostname mismatch should be fixed with correct addressing/certificates rather than disabling verification.

COMMON INTERVIEW TRAP

localhost changes meaning inside a container.

FOLLOW-UP QUESTION

Which diagnostic can show the advertised broker list?

Check DNS and connectivity to every advertised broker hostname and port from inside the pod, TLS/SASL configuration and broker listeners. Successful bootstrap alone is insufficient.

The DLT count spikes after deployment. How do you respond?

THE SIMPLE ANSWER

Classify failures and isolate the rollout regression before replaying or scaling retries.

VISUAL MEMORY MAP

Read left to right

Invalid schema errors → Locate failed boundary → stop incompatible writers

WORKED EXAMPLE

Invalid schema errors -> stop incompatible writers
Transient dependency errors -> bound retries and recover capacity

WHY IT WORKS

Inspect versions, sanitized failure metadata and the first affected deployment. Preserve records, rate-limit recovery and verify a fix on representative payloads. Monitor source, retry and DLT backlogs together.

COMMON INTERVIEW TRAP

Replaying before fixing the cause creates a failure loop.

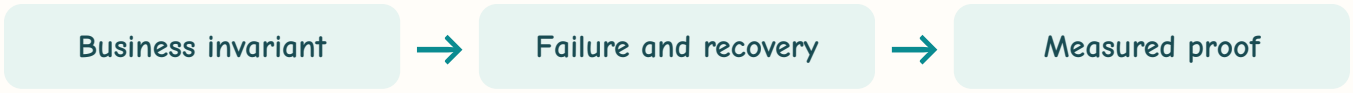
FOLLOW-UP QUESTION

What proves the repaired consumer handles old records?

Pause hazardous replay, preserve samples and deployment evidence, compare schema/settings, fix the consumer and demonstrate old and new records in staging before controlled DLT replay.

20. Senior design and interview challenges

Architect answers name the invariant, mechanism, limits and recovery owner.



Five original interview questions follow, each with its own answer, visual model, worked example, pitfall, and solved follow-up.

Can we guarantee no duplicate email with a Kafka transaction?

THE SIMPLE ANSWER

A Kafka transaction cannot atomically include an arbitrary email provider; use a provider idempotency contract or accept and manage the remaining ambiguity.

VISUAL MEMORY MAP

Read left to right

Send email succeeds → Failure and recovery → crash before checkpoint

WORKED EXAMPLE

Send email succeeds -> crash before checkpoint
Retry can send again unless provider deduplicates

WHY IT WORKS

A durable notification table tracks intent and status but still has a provider-call crash window. State the achievable guarantee honestly and choose an explicit delivery policy for ambiguous outcomes.

COMMON INTERVIEW TRAP

Local sent=true is not an atomic proof of remote delivery.

FOLLOW-UP QUESTION

What if the provider offers no idempotency support?

Without provider idempotency, a timeout after send creates an unknowable duplicate risk. Keep a durable request ledger and accept an explicit delivery tradeoff or human reconciliation.

Can the API return success before send completes?

THE SIMPLE ANSWER

Only if it has already accepted the request through another documented durable mechanism such as an outbox.

VISUAL MEMORY MAP

Read left to right

DB order + outbox committed → Failure and recovery → accepted response

WORKED EXAMPLE

DB order + outbox committed -> accepted response
Async Kafka publication -> monitored completion

WHY IT WORKS

An in-memory future without a durable acceptance record can disappear on process failure. If publishing is the acceptance boundary, handle success, failure and timeout/unknown outcome with an API idempotency policy.

COMMON INTERVIEW TRAP

Fire-and-forget is not durable acceptance.

FOLLOW-UP QUESTION

Which boundary does your HTTP contract promise?

Only if the contract promises durable intent (e.g. a committed outbox row), not confirmed Kafka append or downstream completion. State precisely what success means and offer status tracking.

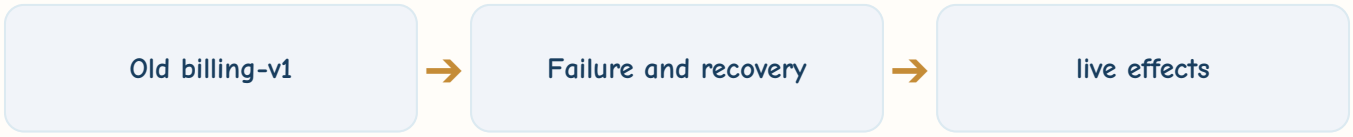
How do you perform a safe consumer-group migration?

THE SIMPLE ANSWER

Define replay starting points, duplicate protection, shadow validation and cutover ownership before launching a new group.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

```
Old billing-v1 -> live effects
  New billing-v2 -> shadow projection -> compare -> cutover
```

WHY IT WORKS

A new group starts according to reset policy unless offsets are deliberately established. Two live groups may both execute external effects. Record the checkpoint and rollback plan so cutover does not silently miss history.

COMMON INTERVIEW TRAP

Renaming groupId is a behavioral change, not a cosmetic one.

FOLLOW-UP QUESTION

How do you choose a consistent cutover point?

Record a source offset or business version watermark, build a shadow group, catch up and compare effects. Freeze/cutover writes briefly if strict order demands an unambiguous boundary.

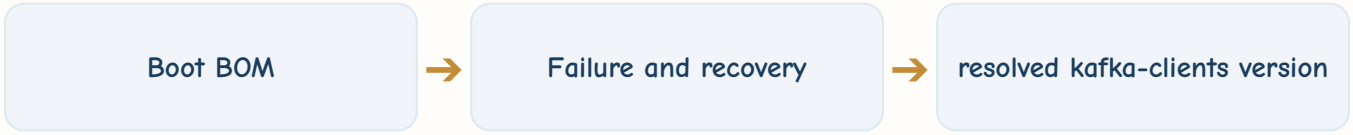
How do you migrate to newer Kafka consumer protocols?

THE SIMPLE ANSWER

Check broker/client compatibility and protocol-specific assignment and timeout settings, then validate rebalance behavior under rollout and failure.

VISUAL MEMORY MAP

Read left to right



WORKED EXAMPLE

Boot BOM -> resolved kafka-clients version
  Broker support + client support -> chosen group.protocol

WHY IT WORKS

Kafka 4.x introduced a new consumer protocol with different management of some timing/assignment options. A Boot 3.5 application's older managed client should not be assumed to expose newer client behavior. Use official migration guidance and staged tests.

COMMON INTERVIEW TRAP

Broker upgrade alone does not change every client API or default.

FOLLOW-UP QUESTION

What would your compatibility matrix include?

List broker/client protocol versions, Spring Kafka/Boot alignment, group protocol feature flags, assignor compatibility and rollback path. Test mixed versions and rolling deployments.

What makes a strong architect-level Kafka interview answer?

THE SIMPLE ANSWER

State the business invariant, chosen mechanism, failure window, recovery process and evidence that it works.

VISUAL MEMORY MAP

Read left to right

Business invariant → Failure and recovery → Measured proof

WORKED EXAMPLE

Invariant: payment applied once per paymentId
Mechanism: provider key + durable ledger
Evidence: timeout, replay, concurrency and crash tests

WHY IT WORKS

Discuss cost, ordering, retention and operational ownership. Separate transport acceptance from business completion. Use measured sizing and acknowledge boundaries rather than claiming exactly-once across every component.

COMMON INTERVIEW TRAP

Naming Kafka, Redis and retries without an invariant is not a design.

FOLLOW-UP QUESTION

Which assumption would you challenge first in this system?

First challenge the required guarantee and the exact business effect: is ordering global or per key, does success mean broker append or payment completed, and which failure can be tolerated?

THE PRACTICE LABORATORY

10 coding & review exercises

Source-based Spring Kafka examples for debugging and testing real durability boundaries.

Write the code → Inject a failure → Verify the durable result

These are Spring/project fragments. A compatible Boot build and a real Kafka/database fixture are needed for broker-level validation. A syntax review is not a live integration test.

C01 · Publish with an explicit confirmation boundary

Send a stable order key and handle both future completion and synchronous failure.
Setup: spring-kafka; String serializers configured

SOURCE WORKED SOLUTION

```
public CompletableFuture<SendResult<String,String>> publish(
       String orderId, String json) {
     try {
       return template.send("orders", orderId, json);
     } catch (RuntimeException failure) {
       return CompletableFuture.failedFuture(failure);
     }
   }
   // Caller observes completion; does not silently discard the result.
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary KafkaTemplate<String,String> is constructor-injected. Test success metadata, exceptionally completed future and a synchronous send exception. An API timeout means unknown outcome, not proof no record exists. Keep a stable eventId for application resends. For durable asynchronous acceptance, use the outbox exercise instead.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C02 · A record listener with durable-work semantics

Invoke the business service synchronously and propagate a processing failure. Setup: spring-kafka; String deserializers; enable-auto-commit=false; ack-mode=record

SOURCE WORKED SOLUTION

```
@Component
   class OrdersListener {
     private final OrderHandler handler;
     OrdersListener(OrderHandler handler) { this.handler = handler; }
     @KafkaListener(topics="orders", groupId="projection-v1")
     public void receive(ConsumerRecord<String,String> record) {
       handler.apply(record.value());
     }
   }
   // OrderHandler is application code: decode, validate and commit effect.
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Use a nontransactional record container for this exercise. No executor handoff and no swallowed exception. Inject a service failure and inspect retry/ recovery and committed offsets. Restart after the durable effect but before the checkpoint, then prove the business effect is idempotent. Do not replace handler.apply with a log-only method and call that end-to-end success.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C03 · Bounded retries and durable quarantine

Retry twice after the initial attempt, then publish to a reviewed DLT. Setup: spring-kafka; nontransactional record container; String payload exercise

SOURCE WORKED SOLUTION

```
@Bean
   DefaultErrorHandler errorHandler(KafkaTemplate<String,String> t) {
     var recoverer = new DeadLetterPublishingRecoverer(t,
         (record, ex) -> new TopicPartition(
             record.topic() + ".DLT", record.partition()));
     recoverer.setFailIfSendResultIsError(true);
     var handler = new DefaultErrorHandler(recoverer,
         new FixedBackOff(1000L, 2L));
     handler.addNotRetryableExceptions(IllegalArgumentException.class);
     return handler;
   }
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Provision orders.DLT with at least the source partition count and authorize its writes. Boot uses this bean with its default container factory; wire it explicitly if using a custom factory. Choose a narrow application validation exception instead of classifying all IllegalArgumentException failures in a real service. Test transient success, permanent rejection, retry exhaustion and DLT send failure. Verify the source progress and recoverer behavior for the effective acknowledgment policy.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C04 · Retry topics with an explicit ordering tradeoff

Use nonblocking retries for independent record work, outside container transactions.
Setup: spring-kafka record listener; retry-topic support enabled; nontransactional template

SOURCE WORKED SOLUTION

```
@Configuration
   @EnableKafkaRetryTopic
   class RetryConfiguration { }


   @RetryableTopic(attempts="3",
       backoff=@Backoff(delay=1000, multiplier=2.0),
       kafkaTemplate="kafkaTemplate")
   @KafkaListener(topics="notifications", groupId="notification-v1")
   public void receive(String message) {
     notifications.sendIdempotently(message);
   }
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary This is a separate alternative to C03, not an extra layer to copy on top. Three attempts includes the first try. Import Backoff from Spring Retry. Supply the notification service and provision/authorize generated retry and DLT topics. Test A failing while B succeeds, and show why strict ordering no longer holds. Do not use this pattern with batch listeners or container transactions in the 3.3 baseline.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C05 · Kafka-only transactional transformation

Consume an input event and publish an output with its offset in one Kafka transaction.
Setup: spring-kafka; Boot default listener factory; read_committed consumers

SOURCE WORKED SOLUTION

```
// application.properties
   spring.kafka.producer.transaction-id-prefix=${INSTANCE_ID}-
   spring.kafka.consumer.enable-auto-commit=false
   spring.kafka.consumer.properties[isolation.level]=read_committed


   // listener method; KafkaTemplate<String,String> injected
   @KafkaListener(topics="order-input", groupId="transform-v1")
   public void transform(String value) {
     template.send("order-output", extractOrderId(value), convert(value));
     // A thrown failure rolls back the container transaction.
   }
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Each active instance needs a unique INSTANCE_ID. Supply extraction/conversion functions, topic provisioning and String serializers/deserializers. Boot configures a KafkaTransactionManager and the default listener container transaction wiring. Verify that custom configuration does not remove it. Assert abort hides output from read_committed and prevents progress advancement. The default rollback processor handles retries; define any recovery policy explicitly. Do not add an HTTP call and assume it rolls back.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C06 · Database inbox deduplication

Atomically claim an eventId and apply the effect without a check-then-insert race. Setup: SQL database; service transaction; PostgreSQL syntax shown

SOURCE WORKED SOLUTION

```
CREATE TABLE processed_event (
     consumer_name varchar(100) NOT NULL,
     event_id varchar(100) NOT NULL,
     PRIMARY KEY (consumer_name, event_id)
   );


   -- Inside the same transaction as the business update:
   INSERT INTO processed_event(consumer_name,event_id)
   VALUES ('balance-v1', :eventId)
   ON CONFLICT DO NOTHING;
   -- If affected rows = 0: duplicate; return without applying effect.
   -- If affected rows = 1: apply validated effect, then COMMIT.
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Use a proxy-invoked @Transactional service and parameterized SQL. Capture the affected-row count, and roll back both operations on business failure. Verify simultaneous duplicate deliveries, transaction rollback, restart and replay. PostgreSQL ON CONFLICT avoids the failed-transaction problem caused by naively catching a uniqueness exception. Other databases need their own reviewed atomic insert technique. Deduplication cleanup must respect replay/retention policy.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C07 · Transactional outbox and publisher crash window

Write order and publication intent atomically, then confirm Kafka before marking delivery. Setup: SQL transaction; scheduled/CDC publisher; illustrative SQL

SOURCE WORKED SOLUTION

```
BEGIN;
   INSERT INTO orders(id,status) VALUES (:orderId,'PENDING');
   INSERT INTO outbox(event_id,aggregate_id,payload,state)
   VALUES (:eventId,:orderId,:json,'PENDING');
   COMMIT;


   -- Publisher state machine, not standalone SQL:
   -- claim pending row with lease / reviewed locking strategy
   -- send stable aggregate key and stable eventId; await confirmation
   -- mark DELIVERED only after successful send
   -- retry unconfirmed rows; tolerate confirmed-but-unmarked duplicates
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Add schemas, indexes, lease expiry, bounded batches and a safe multi-publisher claiming algorithm for the chosen database. Preserve per-aggregate order if required. Crash immediately before send and immediately after broker confirmation: both must leave recoverable intent. Test one business row and one logical effect after duplicate delivery. Monitor oldest pending row age and define archival/cleanup policy. No SQL-only solution removes the post-send marking ambiguity.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C08 · Integration test the durable effect

Use a real broker and database and wait for state instead of sleeping. Setup: spring-kafka-test or Testcontainers; actual persistence implementation

SOURCE WORKED SOLUTION

```
// Test sketch: fixture supplies template, repository and unique topic/group.
   publish(event("e-42", "account-7", 100));
   publish(event("e-42", "account-7", 100));
   await().atMost(Duration.ofSeconds(10)).untilAsserted(() -> {
     assertThat(repository.balance("account-7")).isEqualTo(100);
     assertThat(repository.processedCount("e-42")).isEqualTo(1);
   });
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Awaitility and assertion imports are required; fixture methods are intentionally application-specific. Assert one logical result, not exactly one listener invocation. Use a clean database and wait for listener assignment before publication. Add independent tests for invalid JSON, DLT publication failure, transaction abort and shutdown replay. A test that only mocks repository.balance cannot validate the durable invariant.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C09 · Partition-order exercise

Demonstrate key locality and the limits of asynchronous workers. Setup: broker fixture; fixed partition count; stable producer key

SOURCE WORKED SOLUTION

```
for (int version = 1; version <= 5; version++) {
     template.send("account-events", "account-7",
         "version=" + version).get(10, TimeUnit.SECONDS);
   }
   // Collect ConsumerRecord metadata in a test listener.
   // Assert one partition and increasing offsets for this key.
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary This sequential send loop is a deterministic test, not a throughput publishing pattern. Then introduce bounded asynchronous completion delays and show how side effects can finish out of order despite ordered input. Repeat after an explicitly reviewed partition expansion to discuss why future key placement may change. Never assert a global order for different partitions.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

C10 · Production configuration review

Review a proposed setup and repair its invalid guarantees. Setup: design exercise; no executable setup required

SOURCE WORKED SOLUTION

```
Proposed:
     RF=3, min.insync.replicas=1, acks=1
     consumer group shared by billing and audit
     async worker + listener returns immediately
     @RetryableTopic on transactional batch listener
     Redis flag TTL=10 min prevents all repeated charges
```

CHECKS, LIMITATIONS & FAILURE CASES

Verification and boundary Answer: acks=1 confirms only leader, so propose all acknowledgments and a reviewed minimum ISR/fault-domain policy. Independent services need distinct groups. Do not checkpoint before required worker completion. Retry topics conflict with this transactional/batch baseline; choose a compatible recovery strategy. A Redis flag cannot atomically guard payment; use a stable provider idempotency key and durable ledger. State availability, ordering and replay tradeoffs for each repair.

Focused example: add your project's beans, domain objects, infrastructure and tests. This excerpt is not verified as a standalone application.

FINAL REVISION PLAN

30 days to interview readiness

The schedule follows the source guide. Read, code, explain the error path, and record one check that proves your answer.

Day	Topic	Practice output
01	Event-driven purpose	Acceptance versus completion with an order API
02	Topic and partition model	Draw partitions and record identities
03	Replication and ISR	RF=3 / minimum ISR=2 scenario
04	Boot integration	First application and dependency tree
05	Producer confirmation	C01 and two failure paths
06	Idempotence and producer tuning	Protocol retry vs. business resend
07	Revision checkpoint	Explain Q001-Q025
08	Groups and concurrency	Compare shared vs. independent groups
09	Offsets and acknowledgment	Commit next offset and reset semantics
10	Crash windows	Draw DB effect vs. offset commit
11	Poll budget	Bound processing and timeouts
12	Bounded retries	C03 and DLT publication failure
13	Retry topics	C04 and reordered completion
14	Revision checkpoint	Review Q026-Q045
15	Kafka transactions	C05 and read_committed outcomes
16	Business idempotency	C06 and duplicate delivery
17	Outbox	C07 and confirmed-but-unmarked send
18	Schema evolution	Old/new schema reader combinations
19	Retention and replay	Shadow projection rebuild
20	Integration testing	C08 and durable outcome evidence
21	Revision checkpoint	Review Q046-Q070
22	Observability	Lag age, errors and business alerts
23	Security and networking	Advertised-listener connectivity

Day	Topic	Practice output
24	Streams and binder choices	Windowed aggregation tradeoff
25	Sizing and architecture	Partitions and retained storage estimate
26	Ordering exercise	C09 and partition expansion
27	Configuration review	C10 and safe producer/consumer policy
28	Mock interview A	45-minute implementation round
29	Mock interview B	60-minute order/payment design
30	Final review	Present fault-test evidence

SAFETY AND READINESS

Final interview checklist

ACCEPTANCE

Can I distinguish durable intent, Kafka append, and business completion?

ORDER

Can I state where ordering exists and what retry topics may break?

DUPLICATES

Can I prove the consumer effect is idempotent during replay?

RECOVERY

Can I recover after an outbox relay crash or expired retention?

SECURITY

Can I explain TLS/SASL/ACLs and prevent credential leaks?

OBSERVABILITY

Can I alert on oldest pending event age, not just record counts?

What has been verified

Question titles and original follow-up prompts have been checked against the attachment. The new follow-up answers are an educational expansion. The attached first-demo source and exercise fragments have not been built against downloaded Spring dependencies or tested against a live Kafka cluster.

Publishing rights

Before selling or redistributing this adaptation, confirm you own or have appropriate commercial rights to the source questions, code and diagrams.